

Kotlin 03.2 版 Android 進階功能：透過 MotionLayout 建立動畫

來源：<https://codelabs.developers.google.com/codelabs/motion-layout?hl=zh-tw>

步驟數：12

在本程式碼研究室中，您將使用 MotionLayout 建構含有動態動畫的 Android Kotlin 應用程式。

目錄

1. [事前準備](#)
2. [開始使用](#)
3. [使用 MotionLayout 建立動畫](#)
4. [根據拖曳事件製作動畫](#)
5. [修改路徑](#)
6. [瞭解 keyPositionType](#)
7. [建構複雜路徑](#)
8. [在動作期間變更屬性](#)
9. [變更自訂屬性](#)
10. [拖曳事件和複雜路徑](#)
11. [使用程式碼執行動作](#)
12. [恭喜](#)

1. 事前準備

這個程式碼研究室是「Android Kotlin 進階功能」課程的一部分。建議您依序完成程式碼研究室的作業，但這並非強制要求。所有課程程式碼研究室都列在「[Android Kotlin 進階功能程式碼研究室](#)」到達網頁。

這個程式碼研究室是 Android 動畫系列的一部分。建議您依序完成所有程式碼研究室，因為這些研究室會逐步介紹各項工作。

本系列程式碼研究室包括：

- [3.1 屬性動畫](#)
- [3.2 使用 MotionLayout 的動畫](#)

使用 MotionLayout 製作動畫時，會用到許多與[使用 ConstraintLayout 製作主要畫面格動畫](#)相同的概念，並將這些概念擴展至可精細自訂動畫的程度。

MotionLayout 程式庫可讓您在 Android 應用程式中加入豐富的動態效果。這個程式庫以 ConstraintLayout 為基礎，可讓您為使用 ConstraintLayout 建構的任何項目加入動畫效果。

您可以使用 MotionLayout，同時為多個檢視區塊的位置、大小、顯示設定、Alpha 通道、顏色、高度、旋轉和其他屬性設定動畫效果。運用宣告式 XML，您可以建立彼此協調的動畫。因為這類動畫包含多個檢視畫面，光靠程式碼很難達到相同效果。

動畫是提升應用程式體驗的好方法。動畫的用途如下：

- **顯示變更：**在狀態之間加入動畫效果，讓使用者自然追蹤 UI 中的變更。
- **吸引目光：**使用動畫吸引使用者注意重要的 UI 元素。
- **打造精美設計：**設計中加入有效動作，可讓應用程式看起來更精緻。

必要條件

本程式碼研究室是為具備 Android 開發經驗的開發人員所設計。嘗試完成本程式碼研究室之前，請先：

- 瞭解如何使用 Android Studio 建立含有 Activity 和基本版面配置的應用程式，並在裝置或模擬器上運作執行。熟悉 ConstraintLayout。如要進一步瞭解 ConstraintLayout，請參閱[限制版面配置程式碼研究室](#)。

執行步驟

- 使用 ConstraintSets 和 MotionLayout 定義動畫
- 根據拖曳事件製作動畫
- 使用 KeyPosition 變更動畫
- 使用 KeyAttribute 變更屬性
- 使用程式碼執行動畫
- 使用 MotionLayout 為可收合的標題設定動畫

軟硬體需求

- [Android Studio 4.0](#) (`MotionLayout` 編輯器僅適用於這個版本的 Android Studio 。)
-

2. 開始使用

強烈建議使用最新的 **4.2 Canary** 版，透過 **Motion Editor** 完成本程式碼研究室。

您可以使用 Android Studio 4.0 以上版本完成本程式碼研究室，但可能會在動態效果編輯器中看到不一致的 UI。

[下載](#) 最新版 Android Studio Canary。

如要下載範例應用程式，請執行下列任一動作：

... 或使用下列指令，從指令列複製 GitHub 存放區：

```
$ git clone https://github.com/googlecode labs/motionlayout.git
```

`motionlayout` 存放區包含一個應用程式專案：



- **motionlayout-start**

您在本程式碼研究室中編輯的所有檔案旁，都有一個完成版本，顯示每個步驟的解決方案。

3. 使用 MotionLayout 建立動畫

首先，您要建構動畫，讓檢視區塊在使用者點選時，從畫面頂端開頭移至底部結尾。

請注意，這個步驟的範例程式碼缺少幾個重要元素，您稍後會新增這些元素。

因此，`MotionLayout` 不會正確顯示這個畫面，您會看到空白畫面，或可能遇到異常終止的情況，直到完成這個步驟為止。

如要從範例程式碼建立動畫，您需要下列主要部分：

- `MotionLayout`，(`ConstraintLayout` 的子類別)。您可以在 `MotionLayout` 標記內指定要製作動畫的所有檢視區塊。
- `MotionScene`，也就是描述 `MotionLayout` 動畫的 XML 檔案
- `Transition`，是 `MotionScene` 的一部分，用於指定動畫時間長度、觸發條件，以及檢視畫面的移動方式。
- `ConstraintSet`，指定轉場的開始和結束限制。

讓我們依序瞭解這些項目，首先是 `MotionLayout`。

步驟 1：探索現有程式碼

`MotionLayout` 是 `ConstraintLayout` 的子類別，因此支援所有相同的功能，同時新增動畫。如要使用 `MotionLayout`，請新增 `MotionLayout` 檢視區塊，您會在其中使用 `ConstraintLayout`。

1. 在 `res/layout` 中開啟 `activity_step1.xml`。您會看到 `ConstraintLayout`，其中包含單一星星 `ImageView`，且內部套用了色調。

activity_step1.xml

```
<!-- initial code -->
<androidx.constraintlayout.widget.ConstraintLayout
    ...
    android:layout_width="match_parent"
    android:layout_height="match_parent"
>

    <ImageView
        android:id="@+id/red_star"
        ...
    />

</androidx.constraintlayout.motion.widget.MotionLayout>
```

這個 `ConstraintLayout` 沒有任何限制，因此如果您現在執行應用程式，會看到星星不受限制地顯示，也就是說，星星會出現在不明位置。Android Studio 會針對缺少限制條件發出警告。

步驟 2：轉換為 MotionLayout

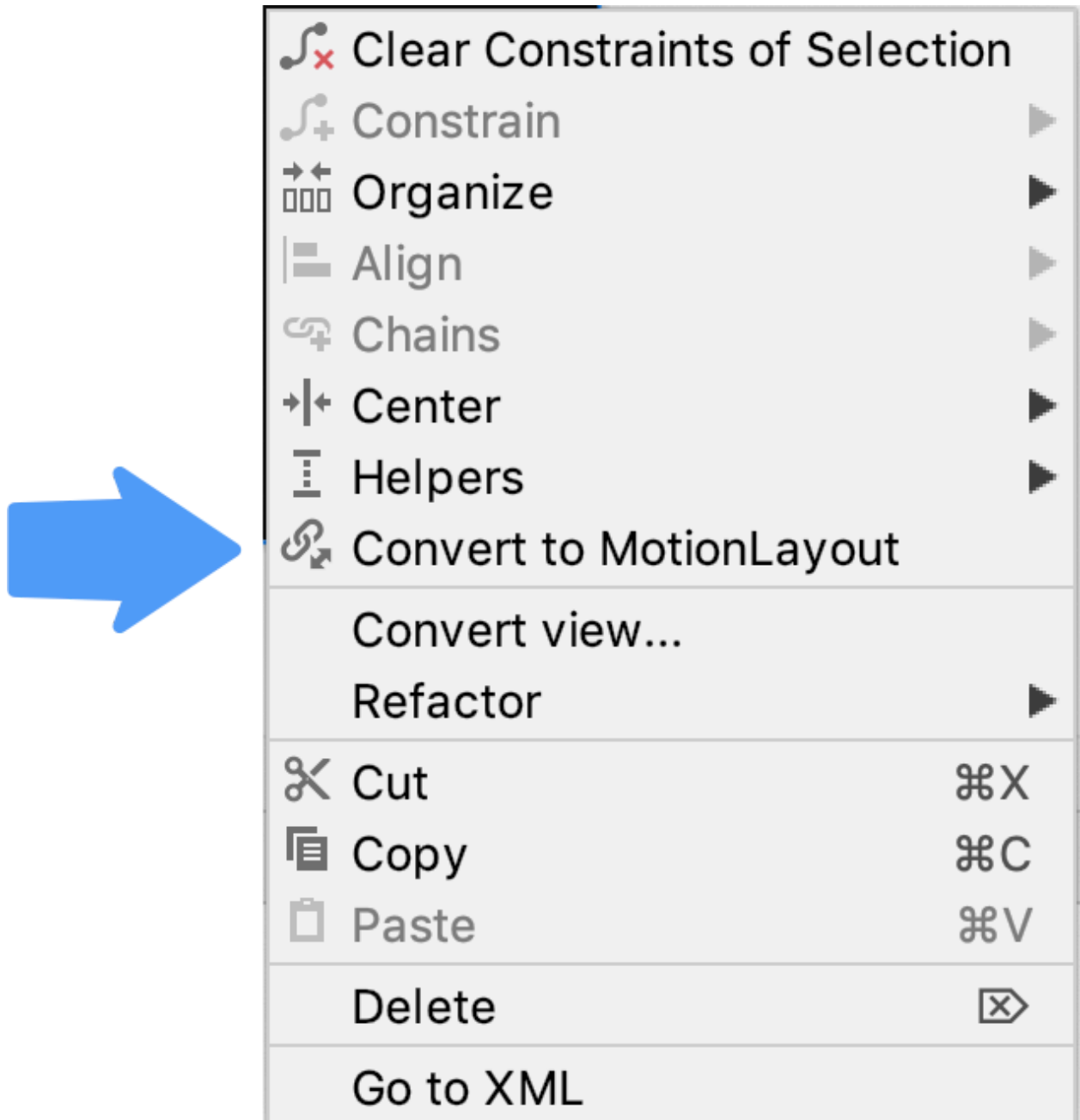
如要使用 `MotionLayout`，製作動畫，必須將 `ConstraintLayout` 轉換為 `MotionLayout`。

如要讓版面配置使用動態場景，必須指向該場景。

1. 如要這麼做，請開啟設計介面。在 Android Studio 4.0 中，查看版面配置 XML 檔案時，使用右上角的「分割」或「設計」圖示，即可開啟設計介面。



2. 開啟設計介面後，在預覽畫面中按一下滑鼠右鍵，然後選取「Convert to MotionLayout」(轉換為 MotionLayout)。



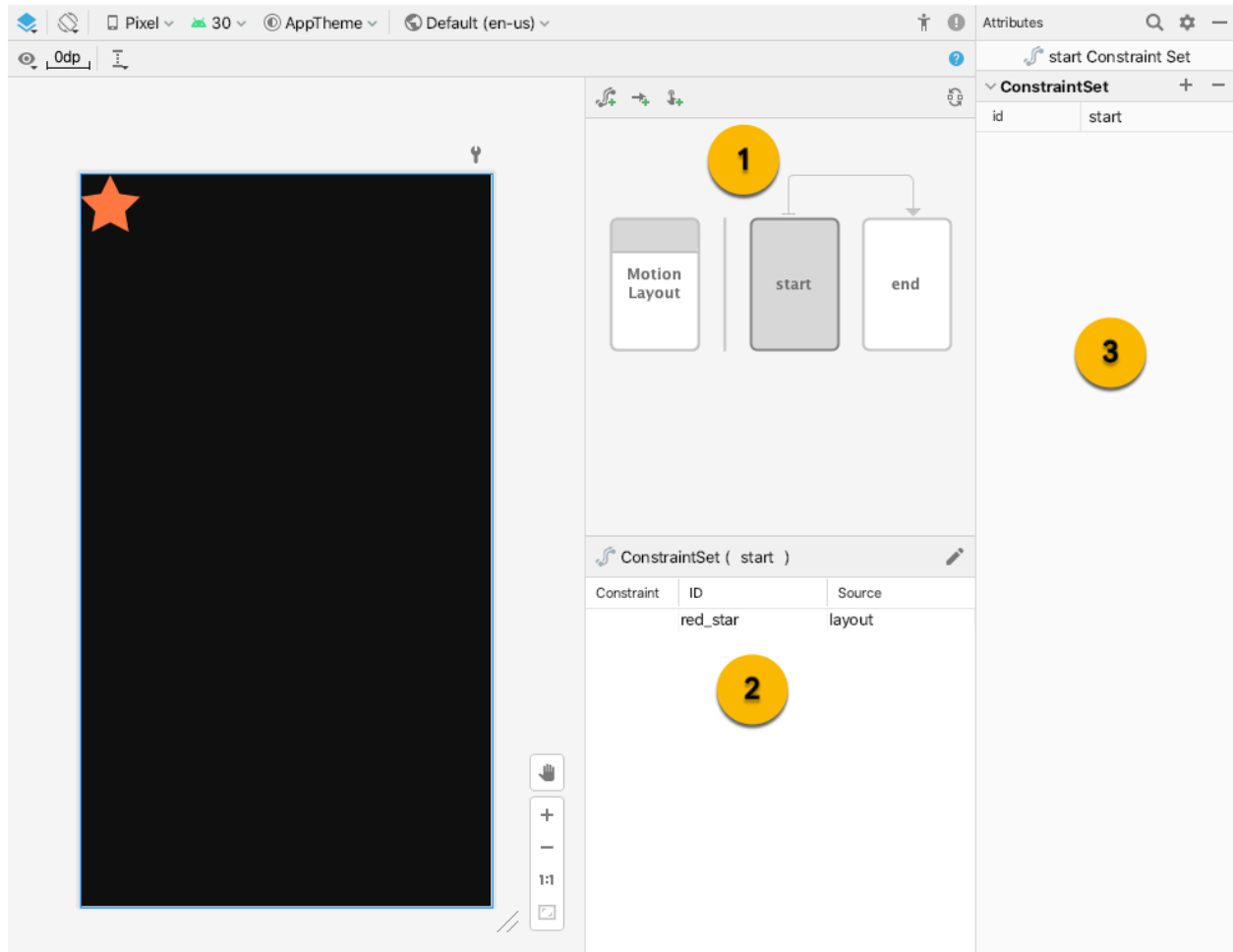
這會將 `ConstraintLayout` 標記替換為 `MotionLayout` 標記，並在 `MotionLayout` 標記中新增指向 `@xml/activity_step1_scene.` 的 `motion:layoutDescription`

activity_step1.xml****

```
<!-- explore motion:layoutDescription="@xml/activity_step1_scene" -->
<androidx.constraintlayout.motion.widget.MotionLayout
    ...
    motion:layoutDescription="@xml/activity_step1_scene">
```

動態場景是單一 XML 檔案，用於描述 `MotionLayout` 中的動畫。

轉換為 `MotionLayout` 後，設計介面會立即顯示 Motion Editor



Motion Editor 中有三個新的 UI 元素：

1. **總覽**：這是強制回應選取畫面，可讓您選取動畫的不同部分。在這張圖片中，`start` `ConstraintSet` 已選取。你也可以點選 `start` 和 `end` 之間的箭頭，選取這兩者之間的轉場效果。
2. **專區**：總覽下方是專區視窗，會根據目前選取的總覽項目而變更。在這張圖片中，選取視窗會顯示 `start` `ConstraintSet` 資訊。
3. **屬性**：屬性面板會顯示目前所選項目的屬性，並允許您從總覽或選取視窗編輯屬性。這張圖片顯示 `start` `ConstraintSet` 的屬性。

步驟 3：定義開始和結束限制

所有動畫都可以定義開始和結束時間。開頭描述動畫播放前的畫面，結尾則描述動畫播放完畢後的畫面。`MotionLayout` 負責計算如何隨時間在開始和結束狀態之間建立動畫效果。

`MotionScene` 會使用 `ConstraintSet` 標記定義開始和結束狀態。`ConstraintSet` 如其名，是一組可套用至檢視區塊的限制。包括寬度、高度和 `ConstraintLayout` 限制。也包含 `alpha` 等屬性。其中不包含檢視畫面本身，只包含這些檢視畫面的限制。

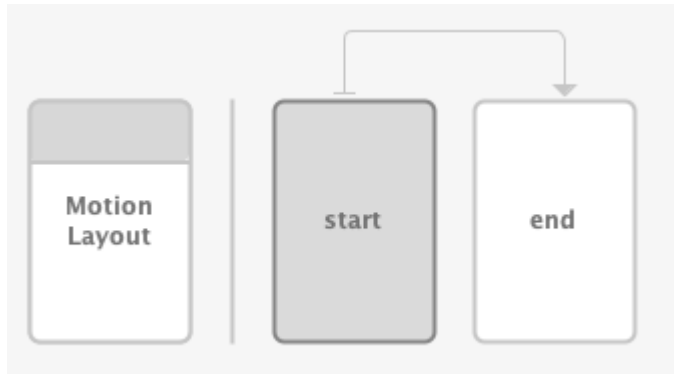
`ConstraintSet` 中指定的任何限制都會覆寫版面配置檔案中指定的限制。如果您在版面配置和 `MotionScene` 中定義限制，系統只會套用 `MotionScene` 中的限制。

限制集只包含限制或版面配置資訊，例如寬度、高度、Alpha 或可見度。也不會包含其他資訊，例如檢視畫面類型或任何檢視畫面專屬屬性，像是 `TextView` 上的文字。

在這個步驟中，您將限制星號檢視區塊從畫面頂端開始，並在畫面底部結束。

如要完成這個步驟，您可以使用 Motion Editor，或直接編輯 `activity_step1_scene.xml` 的文字。

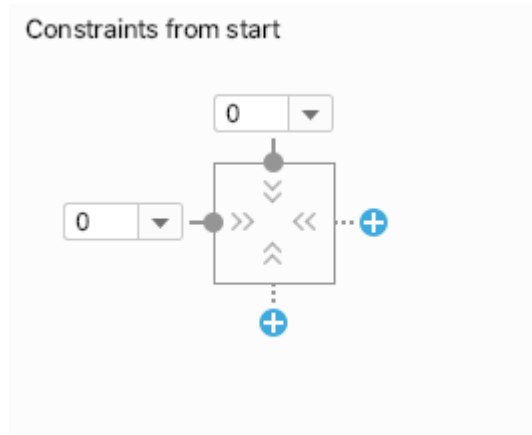
1. 在總覽面板中選取 `start` `ConstraintSet`



2. 在「選取」面板中，選取 `red_star`。目前顯示「來源：`layout`」，表示這個 `ConstraintSet` 沒有限制。使用右上方的鉛筆圖示建立限制條件



3. 在總覽面板中選取 `start` `ConstraintSet` 時，確認 `red_star` 顯示「來源」為 `start`。
4. 在「Attributes」面板中，選取 `start` `ConstraintSet` 中的 `red_star`，然後在頂端新增限制，並按一下藍色的「+」按鈕。



5. 開啟 `xml/activity_step1_scene.xml`，查看 Motion Editor 為這項限制產生的程式碼。

activity_step1_scene.xml

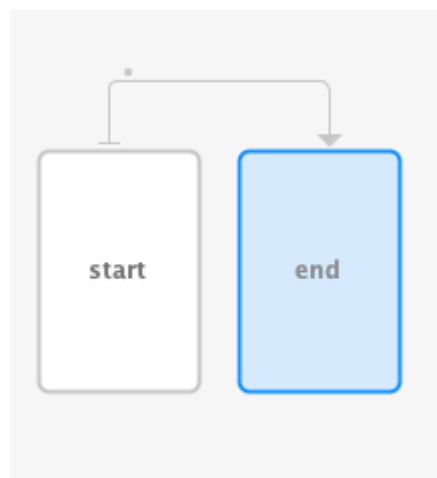
```
<!-- Constraints to apply at the start of the animation -->
<ConstraintSet android:id="@+id/start">
    <Constraint
        android:id="@+id/red_star"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        motion:layout_constraintStart_toStartOf="parent"
        motion:layout_constraintTop_toTopOf="parent" />
</ConstraintSet>
```

`ConstraintSet` 具有 `@id/start` 的 `id`，並指定要套用至 `MotionLayout` 中所有檢視區塊的所有限制。由於這個 `MotionLayout` 只有一個檢視畫面，因此只需要一個 `Constraint`。

`ConstraintSet` 內的 `Constraint` 會指定要限制的檢視區塊 ID，也就是 `activity_step1.xml` 中定義的 `@id/red_star`。請注意，`Constraint` 標記只會指定限制和版面配置資訊。`Constraint` 標記不知道自己套用至 `ImageView`。

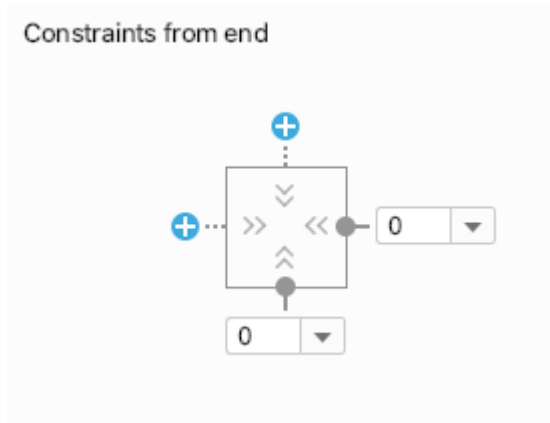
這項限制會指定高度、寬度，以及將 `red_star` 檢視區塊限制在父項頂端開頭所需的另外兩項限制。

1. 在總覽面板中選取 `end` `ConstraintSet`。



2. 按照先前的步驟，在 `end` `ConstraintSet` 中新增 `Constraint` 的 `red_star`。

3. 如要使用 Motion Editor 完成這個步驟，請按一下藍色的「+」按鈕，將限制新增至 `bottom` 和 `end`。



4. XML 中的程式碼如下所示：

activity_step1_scene.xml

```
<!-- Constraints to apply at the end of the animation -->
<ConstraintSet android:id="@+id/end">
    <Constraint
        android:id="@+id/red_star"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        motion:layout_constraintEnd_toEndOf="parent"
        motion:layout_constraintBottom_toBottomOf="parent" />
</ConstraintSet>
```

與 `@id/start` 相同，這個 `ConstraintSet` 在 `@id/red_star` 上也有單一 `Constraint`。這次是將其限制在畫面底部。

您不必將其命名為 `@id/start` 和 `@id/end`，但這樣做很方便。

使用 `MotionLayout` 時，您應在動態場景 XML 檔案中指定動畫檢視區塊的限制，而非在版面配置中指定。如果檢視區塊沒有動畫，可能會在版面配置中受到限制，而不是在動態場景中。

步驟 4：定義轉場效果

每個 `MotionScene` 也必須包含至少一個轉場效果。轉場效果會定義動畫的每個部分，從開始到結束。

轉換必須指定轉換的開始和結束 `ConstraintSet`。轉場效果也可以指定其他動畫修改方式，例如動畫的執行時間長度，或如何透過拖曳檢視區塊來製作動畫。

轉場效果：說明 `MotionScene` 中的一個動畫。

其中包含完整動畫的規格，包括動畫期間的所有變化檢視畫面、動畫應持續的時間，以及如何處理使用者觸控來驅動動畫。 `MotionLayout`

一個 `MotionScene` 可以有多個動畫和多個 `Transition`。

1. 建立 MotionScene 檔案時，動態效果編輯器預設會為我們建立轉場效果。開啟 `activity_step1_scene.xml` 即可查看生成的轉場效果。

activity_step1_scene.xml

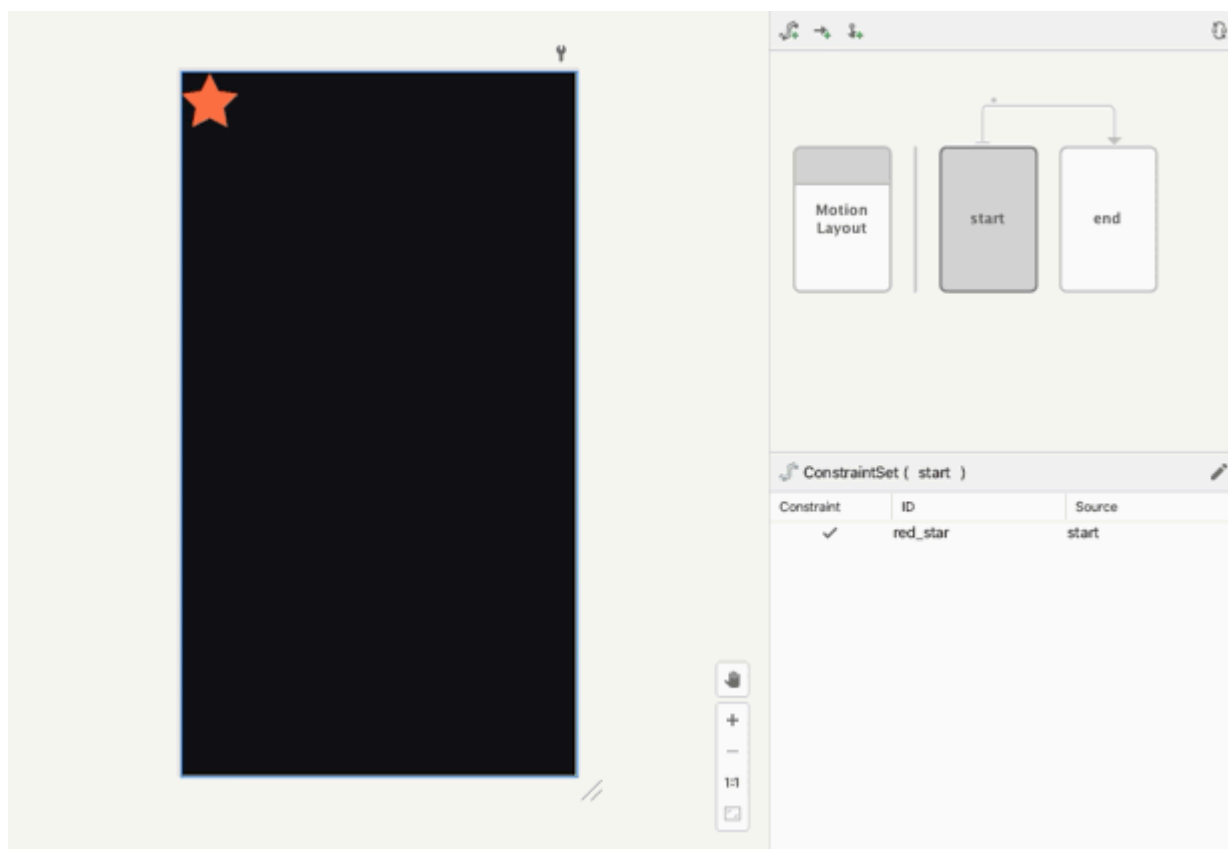
```
<!-- A transition describes an animation via start and end state -->
<Transition
    motion:constraintSetEnd="@+id/end"
    motion:constraintSetStart="@id/start"
    motion:duration="1000">
    <KeyFrameSet>
    </KeyFrameSet>
</Transition>
```

這是 `MotionLayout` 建構動畫所需的一切。查看各個屬性：

- 動畫開始時，系統會將 `constraintSetStart` 套用至檢視區塊。
- 動畫結束時，系統會將 `constraintSetEnd` 套用至檢視區塊。
- `duration` 會指定動畫應播放多長時間 (以毫秒為單位)。

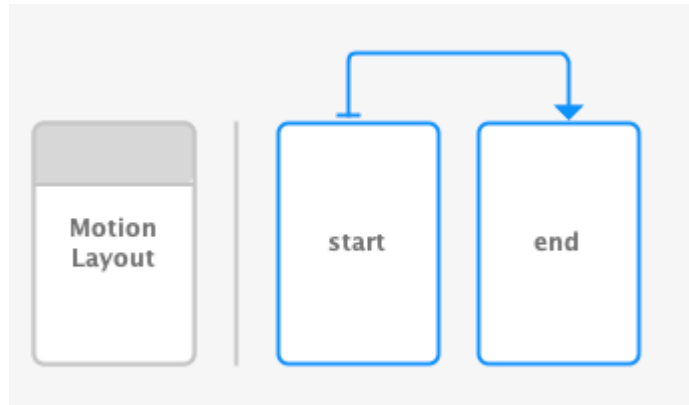
接著，`MotionLayout` 會找出開始和結束限制之間的路徑，並在指定時間內製作動畫。

步驟 5：在 Motion Editor 中預覽動畫

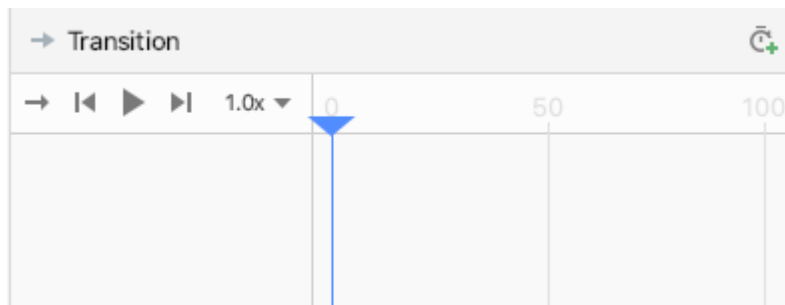


動畫：影片：在 Motion Editor 中播放轉場效果預覽

1. 開啟 Motion Editor，然後在總覽面板中，按一下 `start` 和 `end` 之間的箭頭，選取轉場效果。



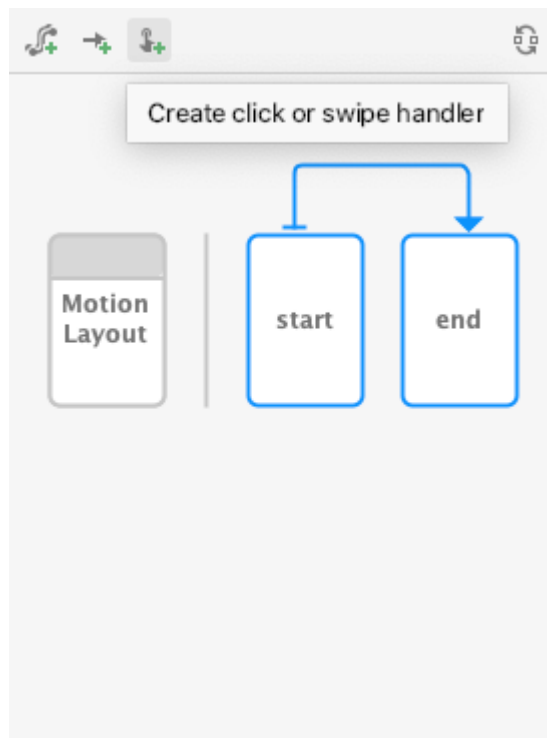
2. 選取轉場效果後，「選取範圍」面板會顯示播放控制項和時間軸。按一下「播放」或拖曳目前位置，即可預覽動畫。



步驟 6：新增點按處理常式

您需要啟動動畫的方法。其中一種做法是讓 `MotionLayout` 回應 `@id/red_star` 上的點擊事件。

1. 開啟動態效果編輯器，然後在總覽面板中，按一下開始和結束之間的箭頭，選取轉場效果。



2. 按一下總覽面板工具列中的「建立點按或滑動處理常式」

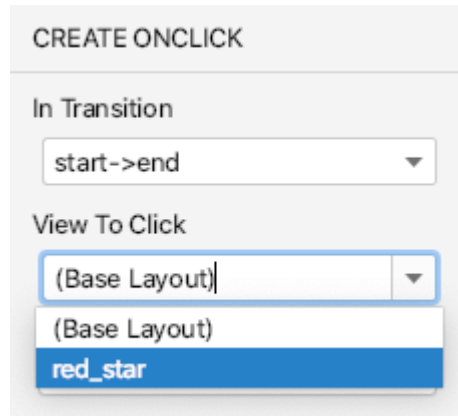


。這會新增處理常式，啟動轉場效果。

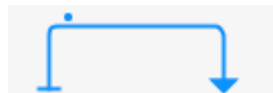
3. 在彈出式視窗中選取「點擊處理常式」



4. 將「View To Click」(可點擊的檢視畫面) 變更為 `red_star`。



5. 按一下「新增」，點擊處理常式會在 Motion Editor 的轉場效果上以小點表示。



6. 在總覽面板中選取轉場效果，然後在屬性面板中，將 `toggle` 的 `clickAction` 屬性新增至您剛才新增的 `OnClick` 處理常式。

▼ OnClick		☑	+	-
clickAction	toggle	▼		
targetId	@+id/red_star	▼		

7. 開啟 `activity_step1_scene.xml`，查看 Motion Editor 生成的程式碼

activity_step1_scene.xml

```
<!-- A transition describes an animation via start and end state -->
<Transition
    motion:constraintSetStart="@+id/start"
    motion:constraintSetEnd="@+id/end"
    motion:duration="1000">
    <!-- MotionLayout will handle clicks on @id/red_star to "toggle" the animation between
the start and end -->
    <OnClick
        motion:targetId="@id/red_star"
        motion:clickAction="toggle" />
</Transition>
```

`Transition` 會告知 `MotionLayout` 使用 `<OnClick>` 標記，在點擊事件發生時執行動畫。查看各個屬性：

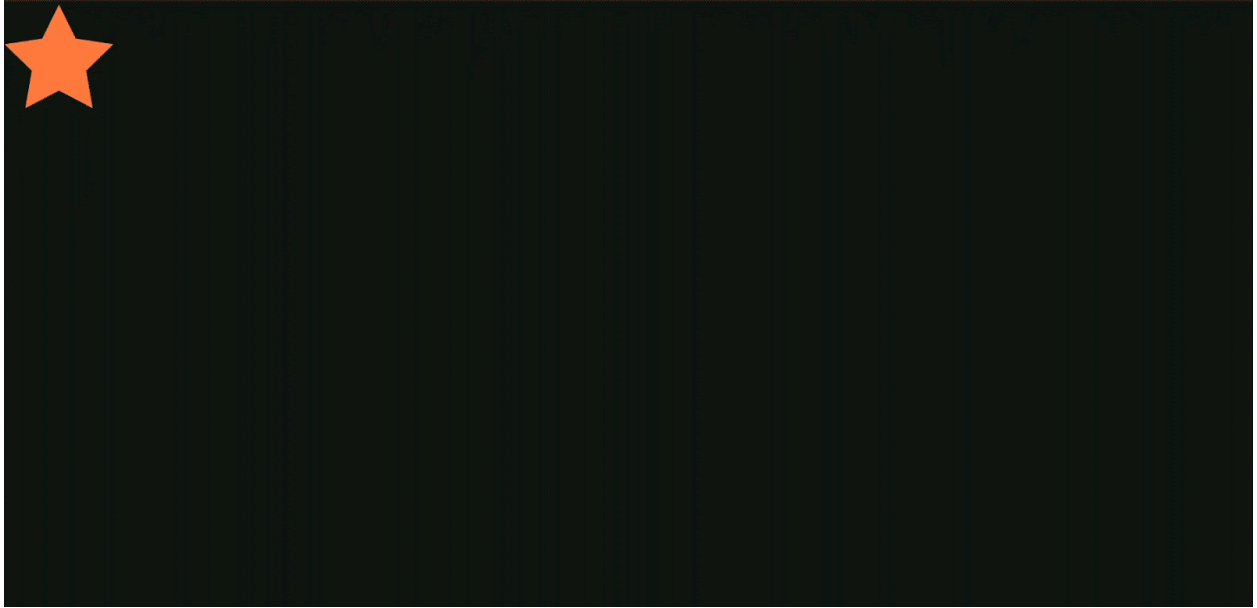
- `targetId` 是要監控點擊次數的檢視畫面。
- `clickAction toggle` 會在點選時切換開始和結束狀態。如要查看 `clickAction` 的其他選項，請參閱[說明文件](#)。

8. 執行程式碼，按一下「步驟 1」，然後按一下紅色星星，即可看到動畫！

您可以將開始或結束位置指向版面配置檔案，而非 `ConstraintSet`，藉此從現有 `ConstraintLayout` 載入 `ConstraintSet`。

步驟 5：實際運用動畫

執行應用程式！按一下星星時，您應該會看到動畫執行。



完成的動態場景檔案會定義一個 `Transition`，指向開始和結束 `ConstraintSet`。

在動畫開始時 (`@id/start`)，星星圖示會限制在畫面頂端。動畫結束時 (`@id/end`)，星星圖示會限制在螢幕底部。

```
<?xml version="1.0" encoding="utf-8"?>

<!-- Describe the animation for activity_step1.xml -->
<MotionScene xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:android="http://schemas.android.com/apk/res/android">
    <!-- A transition describes an animation via start and end state -->
    <Transition
        motion:constraintSetStart="@+id/start"
        motion:constraintSetEnd="@+id/end"
        motion:duration="1000">
        <!-- MotionLayout will handle clicks on @id/star to "toggle" the animation between the
start and end -->
        <OnClick
            motion:targetId="@id/red_star"
            motion:clickAction="toggle" />
    </Transition>

    <!-- Constraints to apply at the end of the animation -->
    <ConstraintSet android:id="@+id/start">
        <Constraint
            android:id="@+id/red_star"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            motion:layout_constraintStart_toStartOf="parent"
            motion:layout_constraintTop_toTopOf="parent" />
    </ConstraintSet>

    <!-- Constraints to apply at the end of the animation -->
    <ConstraintSet android:id="@+id/end">
        <Constraint
            android:id="@+id/red_star"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            motion:layout_constraintEnd_toEndOf="parent"
            motion:layout_constraintBottom_toBottomOf="parent" />
    </ConstraintSet>
</MotionScene>
```

4. 根據拖曳事件製作動畫

最後一個部分介紹了 Motion Layout 和 Motion Editor。本程式碼研究室的其餘步驟將著重於 XML。

您也可以使用 Motion Editor 完成本 Codelabs，Motion Editor 中的新 UI 元素會像這樣在提示中標示出來。

在這個步驟中，您將建構動畫，回應使用者拖曳事件 (使用者滑動螢幕時)，並執行動畫。MotionLayout 支援追蹤觸控事件來移動檢視區塊，以及以物理為基礎的撥動手勢，讓動作流暢。

Motion Editor 目前不支援透過 UI 建立鏈結。替代解決方案如下：

- 左側開始位置連結至父項的開始位置 + 頂端，邊界為 32dp
- 右側星號連結至父項的結尾 + 頂端，邊界為 32 dp
- 紅色星號連結至父項的中心，位於父項頂端，邊界為 32 dp

步驟 1：檢查初始程式碼

1. 如要開始使用，請開啟版面配置檔案 `activity_step2.xml`，其中包含現有的 MotionLayout。請查看程式碼。

activity_step2.xml

```

<!-- initial code -->

<androidx.constraintlayout.motion.widget.MotionLayout
    ...
    motion:layoutDescription="@xml/step2" >

    <ImageView
        android:id="@+id/left_star"
        ...
    />

    <ImageView
        android:id="@+id/right_star"
        ...
    />

    <ImageView
        android:id="@+id/red_star"
        ...
    />

    <TextView
        android:id="@+id/credits"
        ...
        motion:layout_constraintTop_toTopOf="parent"
        motion:layout_constraintEnd_toEndOf="parent"/>
</androidx.constraintlayout.motion.widget.MotionLayout>

```

這個版面配置會定義動畫的所有檢視區塊。三個星號圖示不會受到版面配置限制，因為它們會在動態場景中產生動畫效果。

由於片尾字幕在整個動畫中都停留在相同位置，且不會修改任何屬性，因此套用限制 `TextView`。

如果檢視區塊不是由 `MotionLayout` 動畫製作動畫，則應在版面配置 XML 檔案中指定限制。`MotionLayout` 不會修改 `<MotionScene>` 中未由 `<Constraint>` 參照的限制。

動畫檢視區塊的限制應在動態場景 XML 檔案中設定。

步驟 2：為場景製作動畫

與上一個動畫相同，動畫會由開始和結束 `ConstraintSet`，以及 `Transition` 定義。

定義開始 `ConstraintSet`

1. 開啟動態場景 `xml/step2.xml`，定義動畫。
2. 新增起始限制 `start` 的限制條件。一開始，三顆星星會置中顯示在畫面底部。左右兩側的星星 `alpha` 值為 `0.0`，表示完全透明且隱藏。

step2.xml

```

<!-- TODO apply starting constraints -->

<!-- Constraints to apply at the start of the animation -->
<ConstraintSet android:id="@+id/start">
    <Constraint
        android:id="@+id/red_star"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        motion:layout_constraintStart_toStartOf="parent"
        motion:layout_constraintEnd_toEndOf="parent"
        motion:layout_constraintBottom_toBottomOf="parent" />

    <Constraint
        android:id="@+id/left_star"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:alpha="0.0"
        motion:layout_constraintStart_toStartOf="parent"
        motion:layout_constraintEnd_toEndOf="parent"
        motion:layout_constraintBottom_toBottomOf="parent" />

    <Constraint
        android:id="@+id/right_star"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:alpha="0.0"
        motion:layout_constraintStart_toStartOf="parent"
        motion:layout_constraintEnd_toEndOf="parent"
        motion:layout_constraintBottom_toBottomOf="parent" />
</ConstraintSet>

```

在這個 `ConstraintSet` 中，您要為每顆星指定一個 `Constraint`。動畫開始時，`MotionLayout` 會套用每個限制。

每個星級檢視畫面都使用開始、結束和底部限制，置於畫面底部中央。兩個星號 `@id/left_star` 和 `@id/right_star` 都有額外的 Alpha 值，可讓星號隱形，並在動畫開始時套用。

`start` 和 `end` 限制集會定義動畫的開始和結束。起始限制 (例如 `motion:layout_constraintStart_toStartOf`) 會將某個檢視區塊的起始位置限制為另一個檢視區塊的起始位置。一開始可能會覺得很混淆，因為 `start` 這個名稱同時用於 `和`，而且兩者都用於限制條件的情境。為協助區分兩者，`start` 中的 `layout_constraintStart` 是指檢視區塊的「開頭」，也就是由左至右語言中的左側，以及由右至左語言中的右側。`start` 限制集是指動畫的開頭。

您可以在 `Constraint` 中指定 Alpha 值。這個值會決定檢視區塊的透明度，其中 `"0.0"` 表示完全透明，`"1.0"` 則表示完全不透明。

在兩個 Alpha 值之間製作動畫時，`MotionLayout` 會在這兩個值之間平滑轉換。舉例來說，在 `alpha="0.0"` 和 `alpha="1.0"` 之間建立動畫時，`MotionLayout` 會建立淡入效果。

定義結尾 ConstraintSet

1. 定義結束限制，使用[鏈結](#)將所有三顆星一起放置在 `@id/credits` 下方。此外，這也會將左右星星的 `alpha` 結束值設為 `1.0`。

step2.xml

```
<!-- TODO apply ending constraints -->

<!-- Constraints to apply at the end of the animation -->
<ConstraintSet android:id="@+id/end">

    <Constraint
        android:id="@+id/left_star"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:alpha="1.0"
        motion:layout_constraintHorizontal_chainStyle="packed"
        motion:layout_constraintStart_toStartOf="parent"
        motion:layout_constraintEnd_toStartOf="@id/red_star"
        motion:layout_constraintTop_toBottomOf="@id/credits" />

    <Constraint
        android:id="@+id/red_star"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        motion:layout_constraintStart_toEndOf="@id/left_star"
        motion:layout_constraintEnd_toStartOf="@id/right_star"
        motion:layout_constraintTop_toBottomOf="@id/credits" />

    <Constraint
        android:id="@+id/right_star"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:alpha="1.0"
        motion:layout_constraintStart_toEndOf="@id/red_star"
        motion:layout_constraintEnd_toEndOf="parent"
        motion:layout_constraintTop_toBottomOf="@id/credits" />
</ConstraintSet>
```

最終結果是檢視區塊會從中心向外和向上擴散，並產生動畫效果。

此外，由於 `alpha` 屬性是在 `ConstraintSets` 和 `@id/left_star` 中設定，因此動畫進行時，這兩個檢視區塊都會淡入。`@id/right_star`

根據使用者滑動手勢製作動畫

`MotionLayout` 可以追蹤使用者拖曳事件或滑動動作，建立以物理為基礎的「甩動」動畫。也就是說，如果使用者輕拂檢視區塊，檢視區塊會繼續移動，並像實體物體在表面上滾動時一樣減速。您可以在 `Transition` 中使用 `OnSwipe` 標記新增這類動畫。

1. 將新增 `OnSwipe` 標記的 TODO 替換為 `<OnSwipe motion:touchAnchorId="@id/red_star" />`。

step2.xml

```
<!-- TODO add OnSwipe tag -->

<!-- A transition describes an animation via start and end state -->
<Transition
    motion:constraintSetStart="@+id/start"
    motion:constraintSetEnd="@+id/end">
    <!-- MotionLayout will track swipes relative to this view -->
    <OnSwipe motion:touchAnchorId="@id/red_star" />
</Transition>
```

`OnSwipe` 包含幾個屬性，其中最重要的屬性是 `touchAnchorId`。

- `touchAnchorId` 是會因應觸控操作而移動的追蹤檢視區塊。`MotionLayout` 會讓這個檢視畫面與滑動的手指保持相同距離。
- `touchAnchorSide` 可決定要追蹤檢視畫面的哪一側。如果檢視區塊會調整大小、遵循複雜路徑，或一側移動速度比另一側快，這點就非常重要。
- `dragDirection` 決定這個動畫的相關方向 (上、下、左或右)。

當 `MotionLayout` 監聽拖曳事件時，系統會在 `MotionLayout` 檢視區塊上註冊監聽器，而非 `touchAnchorId` 指定的檢視區塊。使用者在螢幕上任何位置開始手勢時，`MotionLayout` 會保持手指與 `touchAnchorId` 檢視區塊 `touchAnchorSide` 之間的距離恆定。舉例來說，如果使用者在錨定側邊 100dp 處觸控 `MotionLayout`，在整個動畫期間，該側邊都會與手指保持 100dp 的距離。

`OnSwipe` 會監聽 `MotionLayout` 上的滑動動作，而不是 `touchAnchorId` 中指定的檢視區塊。

也就是說，使用者可能會在指定檢視區塊外滑動，以執行動畫。

立即試用

1. 再次執行應用程式，然後開啟「步驟 2」畫面。畫面會顯示動畫。
2. 嘗試「甩動」或在動畫進行到一半時放開手指，探索 `MotionLayout` 如何顯示流暢的物理動畫！



`MotionLayout` 可使用 `ConstraintLayout` 的功能，在差異極大的設計之間製作動畫，創造豐富的效果。

在這個動畫中，所有三個檢視畫面都位於畫面底部的父項，並以父項為基準定位。最後，這三個檢視區塊會以鏈結形式，相對於 `@id/credits` 定位。

儘管版面配置差異極大，`MotionLayout` 仍會在開始和結束之間建立流暢的動畫。

5. 修改路徑

在這個步驟中，您將建構動畫，在動畫期間沿著複雜路徑移動，並在移動期間製作片尾動畫。 `MotionLayout` 可使用 `KeyPosition` 修改檢視區塊在起點和終點之間的路徑。

步驟 1：探索現有程式碼

1. 開啟 `layout/activity_step3.xml` 和 `xml/step3.xml`，查看現有版面配置和動態場景。 `ImageView` 和 `TextView` 會顯示月亮和製作人員名單文字。
2. 開啟動態情境檔案 (`xml/step3.xml`)。您會看到已定義 `Transition` 從 `@id/start` 到 `@id/end` 的 `Transition`。動畫會使用兩個 `ConstraintSets`，將月亮圖片從螢幕左下角移至右下角。月亮移動時，片尾字幕文字會從 `alpha="0.0"` 淡入至 `alpha="1.0"`。
3. 現在執行應用程式，然後選取「步驟 3」。點選月亮後，你會發現月亮會沿著線性路徑 (或直線) 從起點移動到終點。

步驟 2：啟用路徑偵錯

在為月球的移動路徑新增弧線前，建議您在 `MotionLayout` 中啟用路徑偵錯功能。

如要使用 `MotionLayout` 開發複雜動畫，您可以繪製每個檢視區塊的動畫路徑。這項功能有助於將動畫視覺化，並微調動作細節。

您也可以在播放轉場效果時按住 `Control` 鍵，在設計介面中顯示路徑。

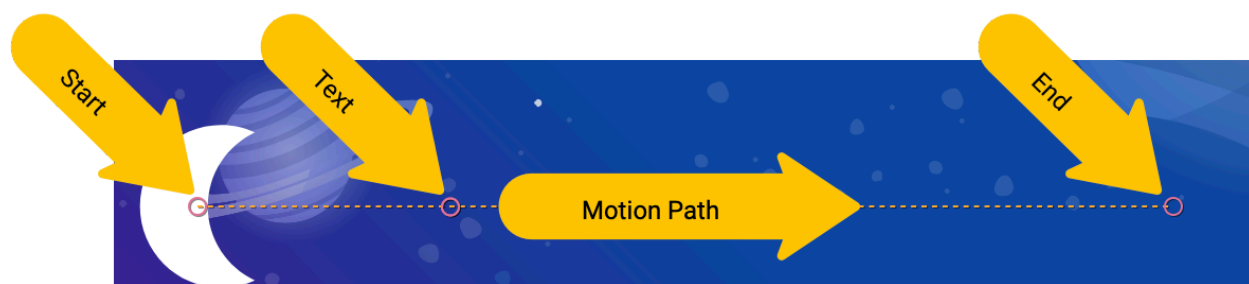
1. 如要啟用偵錯路徑，請開啟 `layout/activity_step3.xml`，並將 `motion:motionDebug="SHOW_PATH"` 新增至 `MotionLayout` 標記。

`activity_step3.xml`

```
<!-- Add motion:motionDebug="SHOW_PATH" -->

<androidx.constraintlayout.motion.widget.MotionLayout
    ...
    motion:motionDebug="SHOW_PATH" >
```

啟用路徑偵錯後，再次執行應用程式時，您會看到所有檢視區塊的路徑都以虛線顯示。



- 圓圈代表一個檢視區塊的開始或結束位置。
- 線條代表單一檢視區塊的路徑。
- 菱形代表會修改路徑的 `KeyPosition`。

舉例來說，在這部動畫中，中間的圓圈是片尾文字的位置。

步驟 3：修改路徑

`MotionLayout` 中的所有動畫都是由開始和結束 `ConstraintSet` 定義，可定義動畫開始前和完成後的畫面。根據預設，`MotionLayout` 會在位置變更的每個檢視區塊的開始和結束位置之間，繪製線性路徑 (直線)。

如要建構複雜路徑 (例如本範例中的月亮弧線)，`MotionLayout` 會使用 `KeyPosition` 修改檢視區塊在開始和結束之間的路徑。

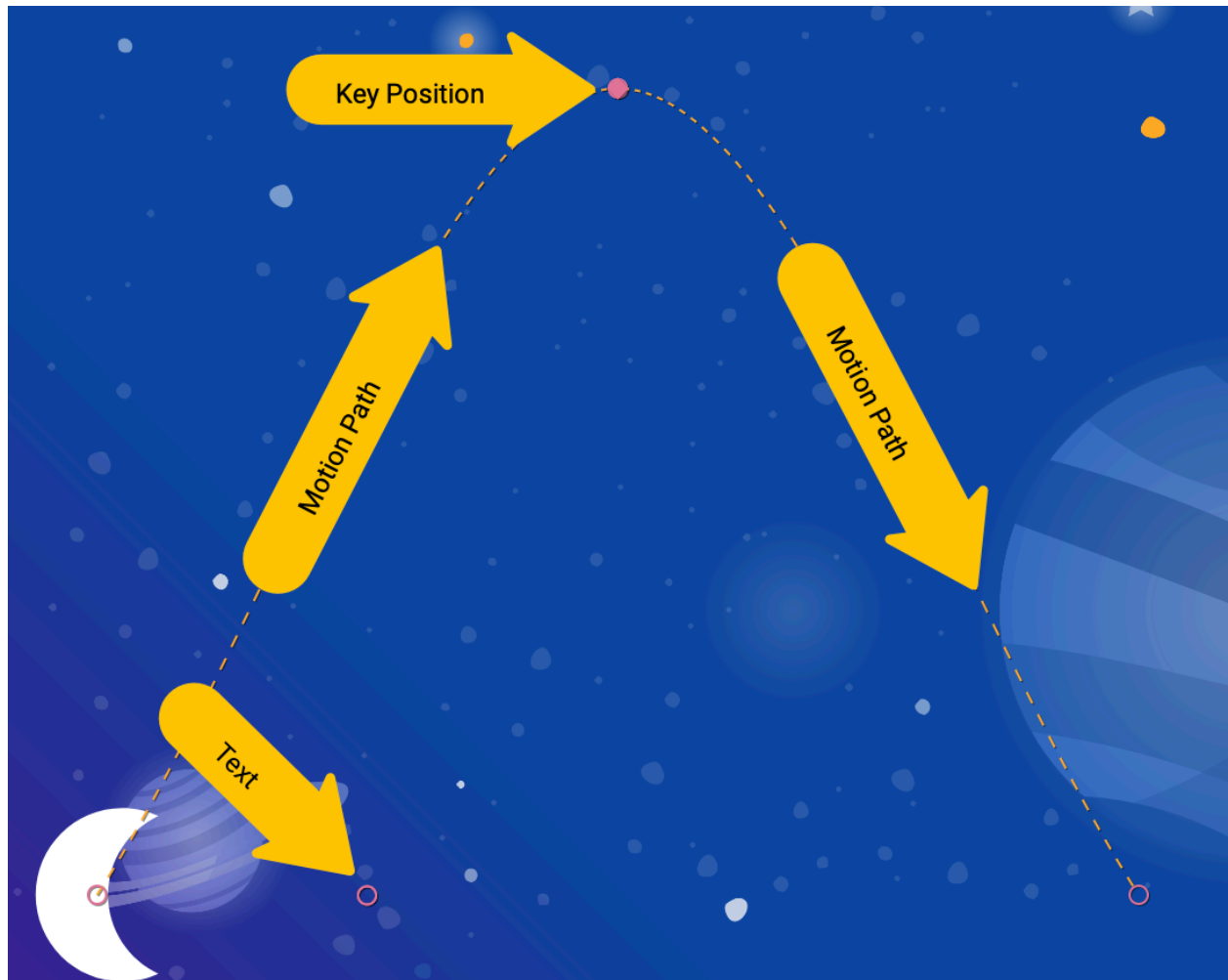
`KeyPosition` 會修改檢視畫面在開始和結束 `ConstraintSet` 之間的路徑。

這會扭曲檢視路徑，使其通過起點和終點之間的第三個 (或第四個、第五個等) 點。甚至可以沿著 X 軸或 Y 軸加快或放慢進度。

`KeyPosition` 只能在動畫期間變更路徑，無法變更起點或終點。

`ConstraintSet` 一律會指定動畫開始和結束時的檢視區塊最終位置。

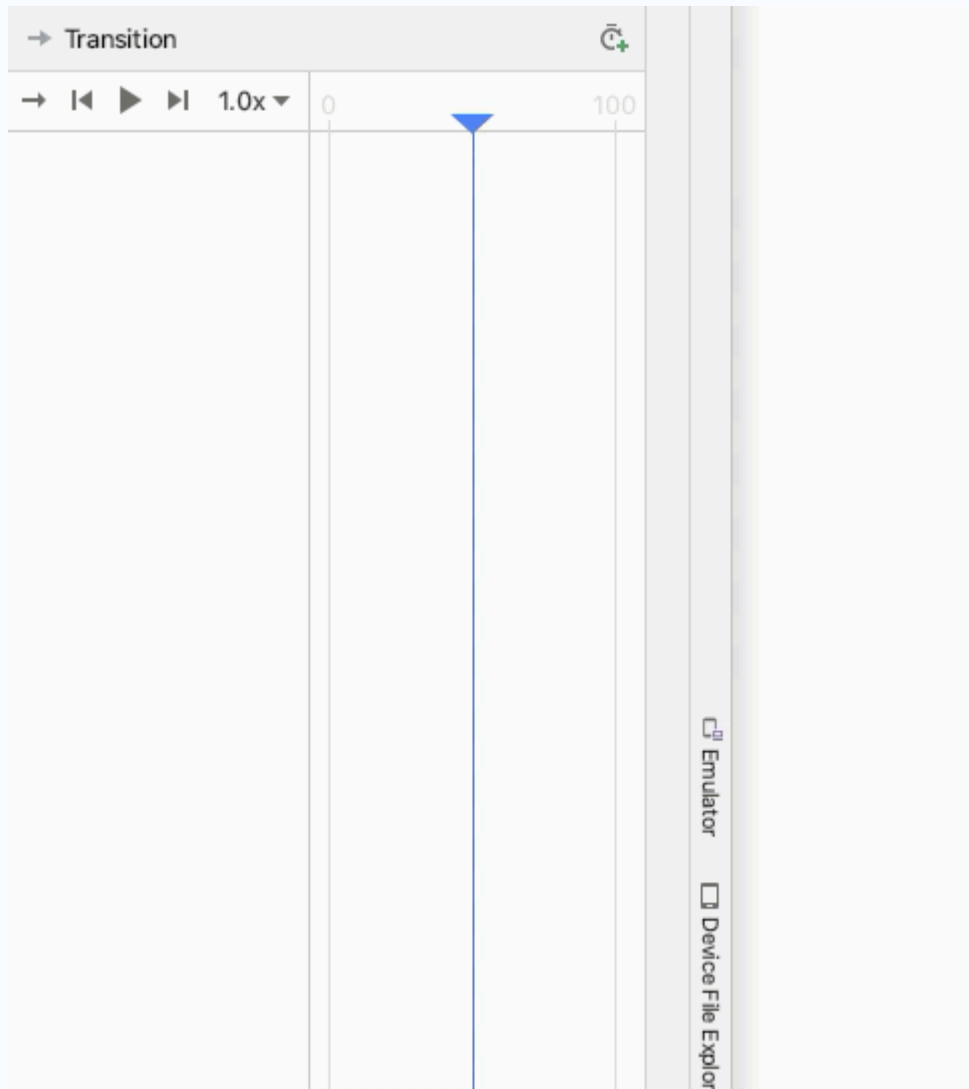
1. 開啟 `xml/step3.xml`，並將 `KeyPosition` 新增至場景。`KeyPosition` 標記會放在 `Transition` 標記內。



step3.xml

```
<!-- TODO: Add KeyFrameSet and KeyPosition -->
<KeyFrameSet>
  <KeyPosition
    motion:framePosition="50"
    motion:motionTarget="@id/moon"
    motion:keyPositionType="parentRelative"
    motion:percentY="0.5"
  />
</KeyFrameSet>
```

如要在 Motion Editor 中完成這個步驟，請在總覽面板中選取轉場效果，然後觀看這部影片。



`KeyFrameSet` 是 `Transition` 的子項，也是一組應在轉場期間套用的所有 `KeyFrames`，例如 `KeyPosition`。

`MotionLayout` 會計算月球在起點和終點之間的路徑，並根據 `KeyFrameSet` 中指定的 `KeyPosition` 修改路徑。再次執行應用程式，即可查看路徑的修改方式。

`KeyPosition` 具有多個屬性，可說明路徑的修改方式。最重要的包括：

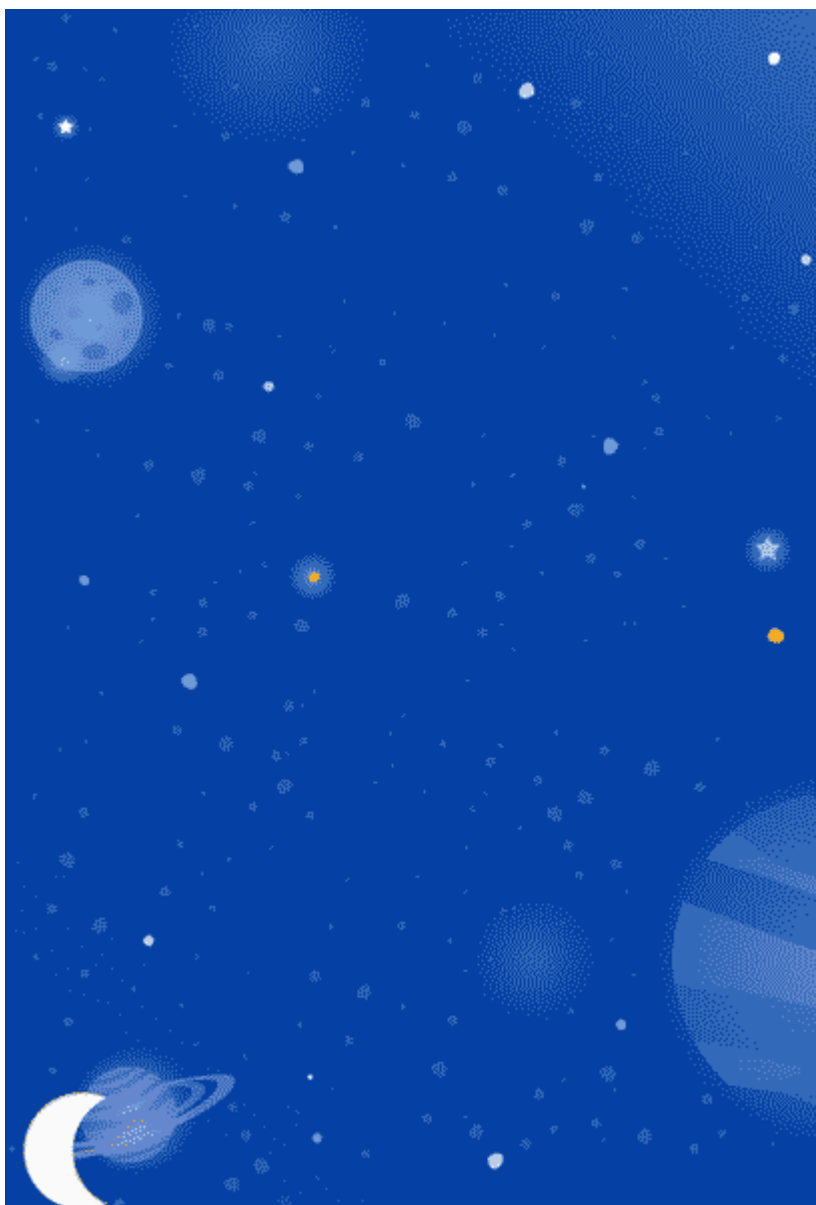
- `framePosition` 是介於 0 到 100 之間的數字。定義動畫中套用 `KeyPosition` 的時間點，其中 1 代表動畫的 1%，99 則代表動畫的 99%。因此，如果值為 50，請套用在正中間。
- `motionTarget` 是這個 `KeyPosition` 修改路徑的檢視畫面。
- `keyPositionType` 是這個 `KeyPosition` 修改路徑的方式。可以是 `parentRelative`、`pathRelative` 或 `deltaRelative` (如後續步驟所述)。
- `percentX` | `percentY` 是在 `framePosition` 修改路徑的程度 (值介於 0.0 和 1.0 之間，允許負值和 >1 的值)。

您可以這樣解讀：「在 `framePosition`，根據 `keyPositionType` 決定的座標，將 `motionTarget` 的路徑修改為移動 `percentX` 或 `percentY`。」

根據預設，`MotionLayout` 會將修改路徑後產生的任何邊角設為圓角。查看剛建立的動畫，您會發現月亮在彎道處沿著彎曲路徑移動。在大多數動畫中，這就是您要的結果，如果不是，您可以指定 `curveFit` 屬性來自訂。

馬上試試

再次執行應用程式，您會看到這個步驟的動畫。



月亮會沿著弧線移動，因為它會經過 `Transition` 中指定的 `KeyPosition`。

```
<KeyPosition
    motion:framePosition="50"
    motion:motionTarget="@id/moon"
    motion:keyPositionType="parentRelative"
    motion:percentY="0.5"
/>
```

您可以將這段程式碼解讀為：「在 `framePosition 50` (動畫播放到一半時)，根據 `parentRelative` (整個 `MotionLayout`) 決定的座標，將 `motionTarget @id/moon` 的路徑移動 `50% Y` (螢幕的一半高度)。」`KeyPosition` 因此，在動畫播放到一半時，月亮必須通過螢幕上 `50%` 的 `KeyPosition`。這個 `KeyPosition` 完全不會修改 `X` 動作，因此月亮仍會從頭到尾水平移動。`MotionLayout` 會找出通過這個 `KeyPosition` 的平滑路徑，並在起點和終點之間移動。

仔細觀察會發現，片尾字幕的文字會受到月亮位置的限制。為什麼不會上下移動？



```
<Constraint
    android:id="@id/credits"
    ...
    motion:layout_constraintBottom_toBottomOf="@id/moon"
    motion:layout_constraintTop_toTopOf="@id/moon"
/>
```

結果顯示，即使修改月球的路徑，月球的起點和終點位置也不會垂直移動。 `KeyPosition` 不會修改開始或結束位置，因此片尾文字會限制在月球的最終結束位置。

如果希望片尾字幕隨著月亮移動，可以在片尾字幕中加入 `KeyPosition`，或修改 `@id/credits` 的開始限制。

`KeyPosition` 絕不會修改開始和結束限制。

即使檢視區塊是沿著複雜的動態路徑移動 (例如 `@id/moon`)，開始和結束限制條件仍會停留在初始或最終位置。其他受限於 `@id/moon` 的檢視區塊不會遵循中間的路徑。

在下一節中，您將深入瞭解 `MotionLayout` 中的不同類型 `keyPositionType`。

6. 瞭解 keyPositionType

在上一個步驟中，您使用了 `keyPosition` 類型的 `parentRelative`，將路徑偏移了螢幕的 50%。 `keyPositionType` 屬性會決定 `MotionLayout` 根據 `percentX` 或 `percentY` 修改路徑的方式。

```
<KeyFrameSet>
  <KeyPosition
    motion:framePosition="50"
    motion:motionTarget="@id/moon"
    motion:keyPositionType="parentRelative"
    motion:percentY="0.5"
  />
</KeyFrameSet>
```

`keyPosition` 可能有三種不同類型：`parentRelative`、`pathRelative` 和 `deltaRelative`。指定類型會變更 `percentX` 和 `percentY` 的計算座標系統。

什麼是座標系統？

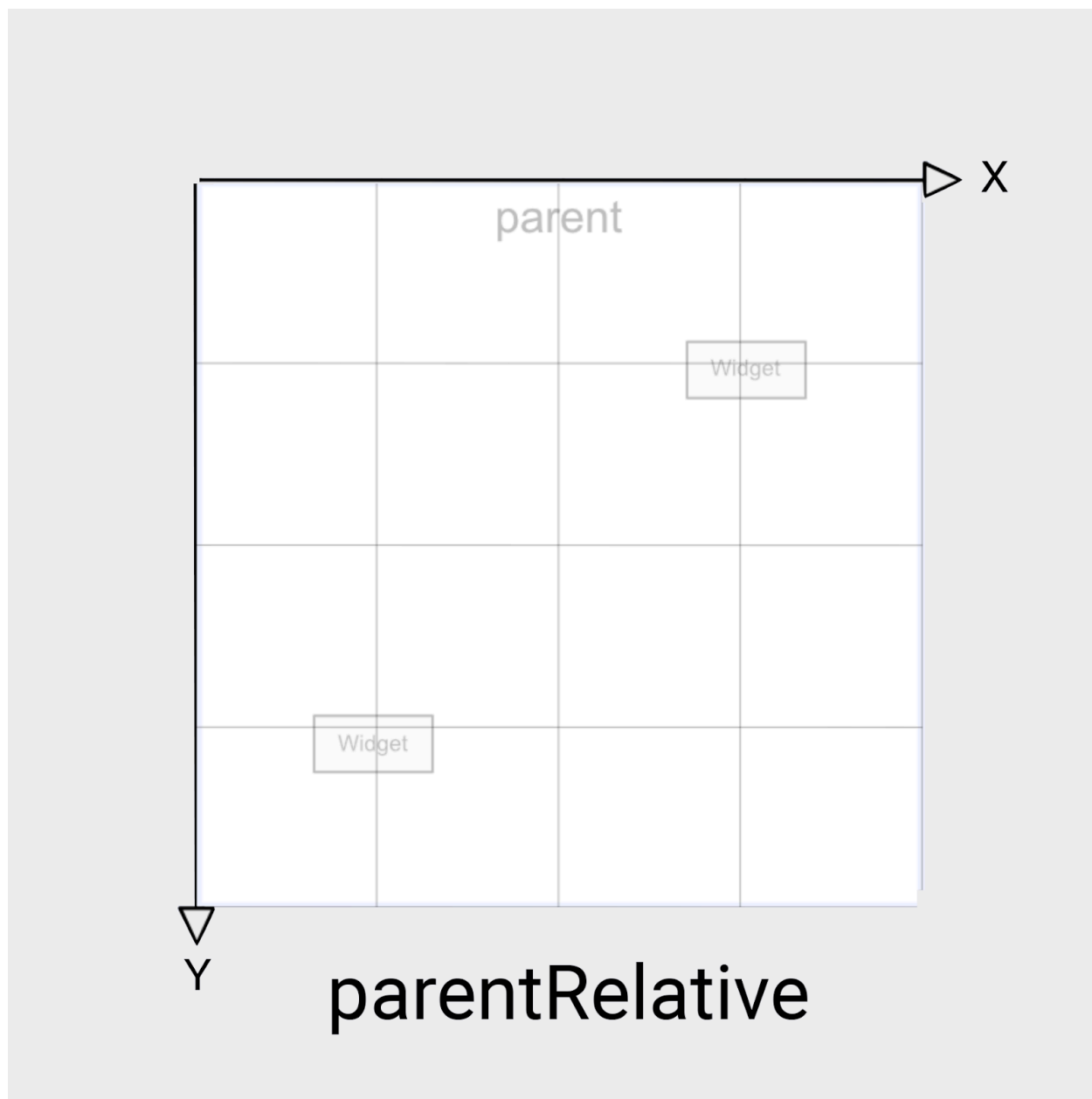
[座標系統](#)可指定空間中的點。這類標籤也可用於說明畫面上的位置。

`MotionLayout` 座標系統是[笛卡兒座標系統](#)。也就是說，這類圖表有兩條垂直線定義的 X 軸和 Y 軸。兩者的主要差異在於螢幕上 X 軸的位置 (Y 軸一律垂直於 X 軸)。

`MotionLayout` 中的所有座標系統在 X 軸和 Y 軸上都使用介於 0.0 和 1.0 之間的值。可使用負值，以及大於 1.0 的值。舉例來說，`percentX` 值為 -2.0，表示要朝 X 軸的反方向移動兩次。

如果覺得這些內容太像代數課，請參考下方的圖片！

parentRelative 座標

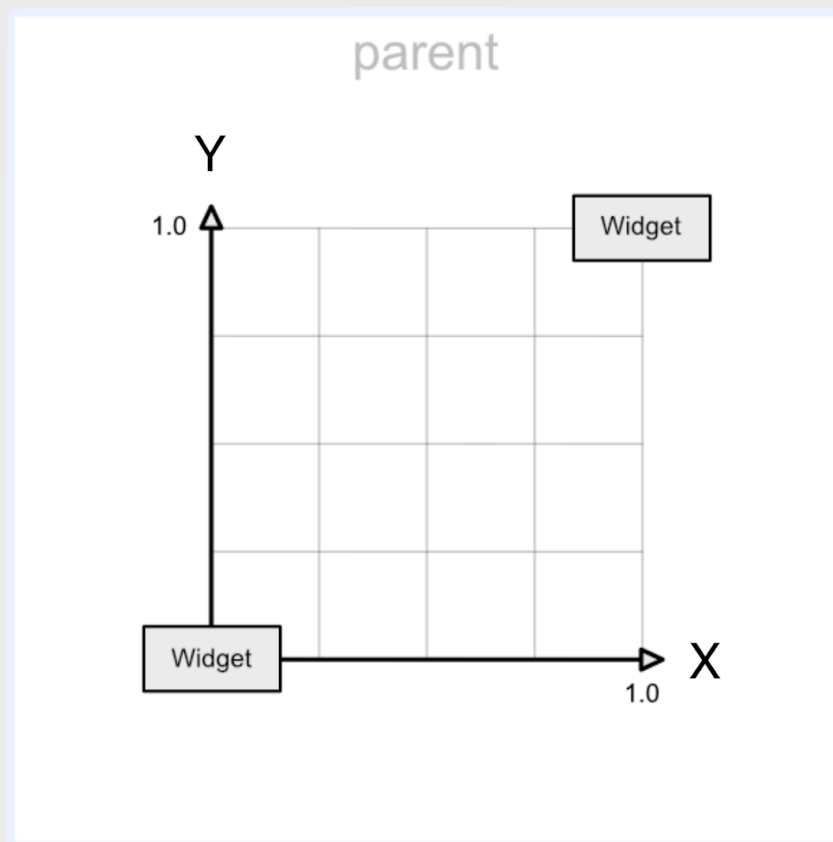


`keyPositionType` 的 `parentRelative` 使用與螢幕相同的座標系統。定義整個 `MotionLayout` 的左上角 `(0, 0)`，以及右下角 `(1, 1)`。

如要製作動畫，讓動畫在整個 `MotionLayout` 中移動 (例如本例中的月亮弧線)，請使用 `parentRelative`。

不過，如果想修改相對於動作的路徑 (例如稍微彎曲)，其他兩個座標系統會是更好的選擇。

deltaRelative 座標



deltaRelative

Delta 是數學用語，代表變化，因此 `deltaRelative` 的意思是「相對變化」。`deltaRelative` 座標 $(0,0)$ 是檢視區塊的起始位置， $(1,1)$ 則是結束位置。X 軸和 Y 軸會與螢幕對齊。

X 軸一律為螢幕上的水平軸，Y 軸一律為螢幕上的垂直軸。與 `parentRelative` 相比，主要差異在於座標只會描述檢視區塊移動的螢幕部分。

`deltaRelative` 是絕佳的座標系統，可單獨控制水平或垂直動作。舉例來說，您可以建立動畫，在 50% 時完成垂直 (Y) 移動，然後繼續水平 (X) 動畫。

`deltaRelative` 中的座標是相對於檢視區塊的動作。

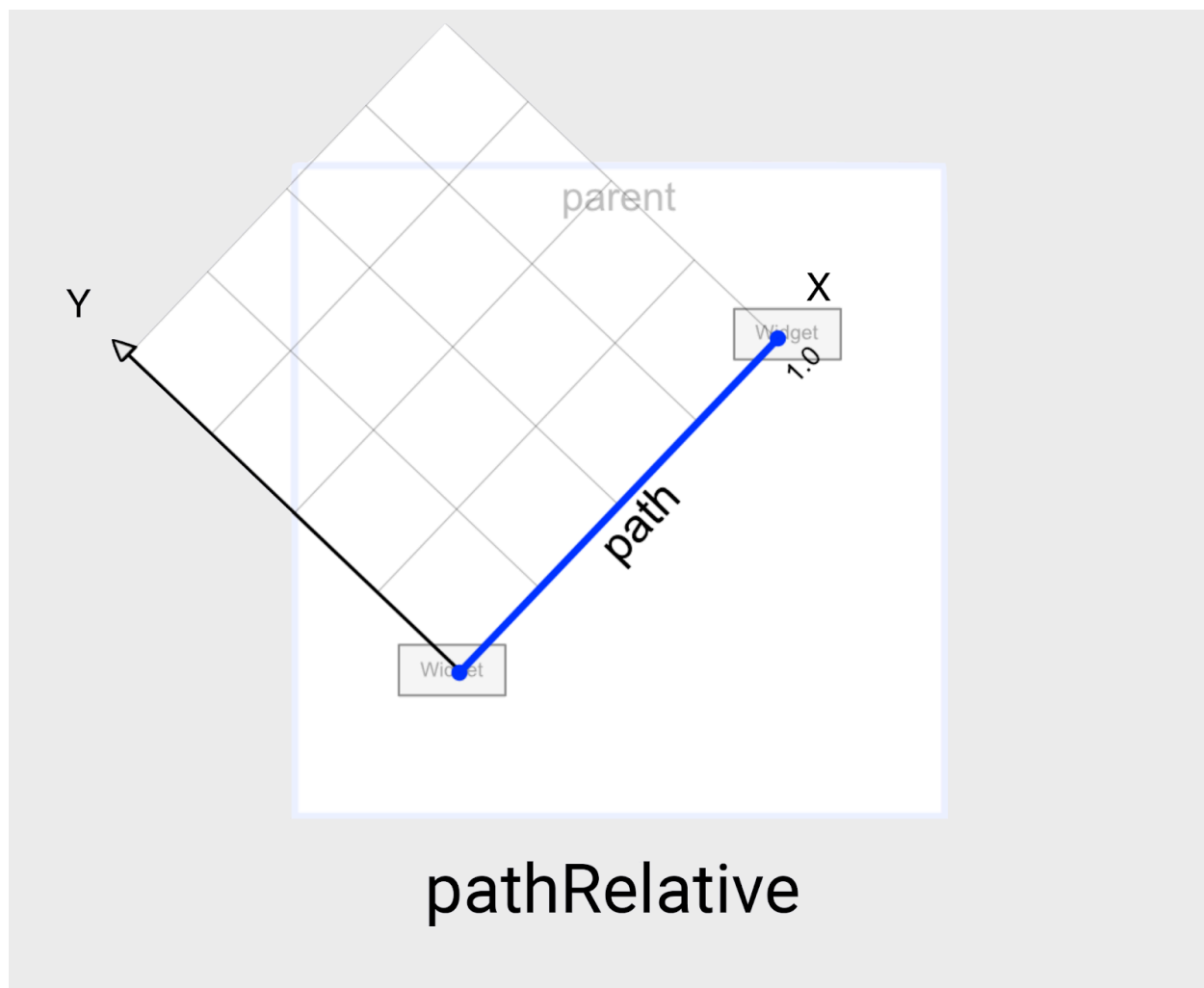
因此，如果檢視畫面向右移動，X 也會向右移動。如果檢視畫面向左移動，X 就會向左移動。同樣地，Y 維度會跟隨檢視區塊上下移動。

當然，如果希望檢視畫面彈跳超過開始或停止位置，隨時可以使用負值或大於 1.0 的值！

`deltaRelative` 的座標會根據檢視區塊在 X 和 Y 方向移動的距離擴大或縮小。

因此，如果檢視畫面在某個方向上移動幅度不大或完全沒有移動，`deltaRelative` 就不會在該方向上建立實用的座標系統。舉例來說，如果檢視區塊從右向左水平移動，`deltaRelative` 中的 Y 軸就不會很有用。

pathRelative 座標



`MotionLayout` 中的最後一個座標系統是 `pathRelative`。這與其他兩者大不相同，因為 X 軸會沿著動態路徑從頭到尾移動。因此 $(0,0)$ 是起始位置， $(1,0)$ 則是結束位置。

為什麼需要這項功能？乍看之下，這相當令人驚訝，尤其是因為這個座標系統甚至未與螢幕座標系統對齊。

結果發現 `pathRelative` 其實有幾種用途。

- 在動畫的某個部分加快、減慢或停止檢視畫面。由於 X 維度一律會與檢視畫面採取的路徑完全相符，因此您可以使用 `pathRelative KeyPosition` 變更路徑中特定點的到達時間。`framePosition` 因此，`KeyPosition` 的 `framePosition="50"` 為 `percentX="0.1"`，會導致動畫花費 50% 的時間，移動前 10% 的動作。
- 在路徑中新增細微的弧形。由於 Y 維度一律垂直於動作，因此變更 Y 會改變路徑，使路徑相對於整體動作彎曲。
- 新增第二個維度時，`deltaRelative` 不會運作。如果是完全水平和垂直的動作，`deltaRelative` 只會建立一個實用維度。不過，`pathRelative` 一律會建立可用的 X 和 Y 座標。

請注意，`pathRelative` 一律會根據動作定義座標系統，而非螢幕。

也就是說，它可能與整體畫面呈某個角度。如要根據螢幕座標修改水平或垂直動作，建議選擇其他座標系統。

在下一個步驟中，您將瞭解如何使用多個 `KeyPosition` 建構更複雜的路徑。

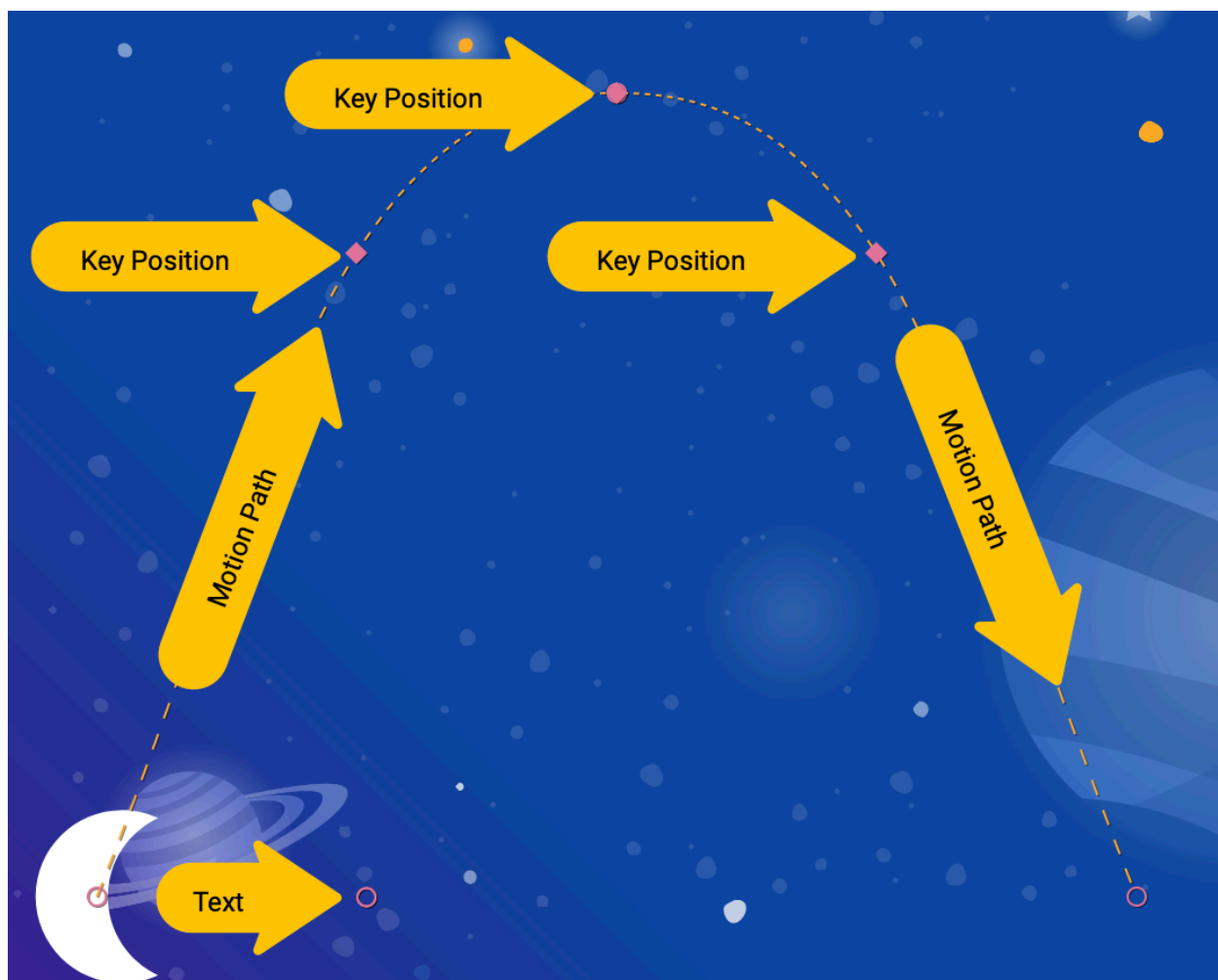
7. 建構複雜路徑

看看您在上一個步驟中建立的動畫，雖然確實會產生平滑曲線，但形狀可以更像「月亮」。

使用多個 `KeyPosition` 元素修改路徑

`MotionLayout` 可以視需要定義任意數量的 `KeyPosition`，進一步修改路徑，取得任何動作。您將為這個動畫建構弧形，但如果想要，也可以讓月亮在畫面中央上下跳動。

1. 開啟 `xml/step4.xml`。您會發現這個檢視區塊與上一個步驟中新增的檢視區塊具有相同的檢視畫面和 `KeyFrame`。
2. 如要將曲線頂端設為圓弧狀，請在 `@id/moon` 的路徑中新增兩個 `KeyPositions`，一個在到達頂端之前，另一個在到達頂端之後。



step4.xml

```
<!-- TODO: Add two more KeyPositions to the KeyFrameSet here -->
<KeyPosition
    motion:framePosition="25"
    motion:motionTarget="@id/moon"
    motion:keyPositionType="parentRelative"
    motion:percentY="0.6"
/>
<KeyPosition
    motion:framePosition="75"
    motion:motionTarget="@id/moon"
    motion:keyPositionType="parentRelative"
    motion:percentY="0.6"
/>
```

這些 `KeyPositions` 會在動畫進行 25% 和 75% 時套用，並導致 `@id/moon` 沿著螢幕頂端 60% 的路徑移動。結合現有的 `KeyPosition` 50%，即可為月亮建立平滑的移動弧線。

在 `MotionLayout` 中，您可以視需要新增任意數量的 `KeyPositions`，取得想要的動態路徑。`MotionLayout` 會在指定的 `framePosition` 套用每個 `KeyPosition`，並找出如何建立流暢的動作，讓所有 `KeyPositions` 都能順利完成。

`KeyPosition` 一律會定義 (`x`, `y`, `width`, `height`) 位置，`View` 在從開始到結束的轉換期間必須經過該位置。

如果您未指定一個維度，系統會採用預設值 (零，或根據 `framePosition` 的適當值)。在本範例中，只指定 `percentY` 時，`percentX` 會保留該 `keyPosition` 的預設值。

馬上試試

1. 再次執行應用程式。前往**步驟 4**，查看實際動畫。點選月亮後，月亮會沿著路徑從起點移動到終點，並經過 `KeyFrameSet` 中指定的每個 `KeyPosition`。

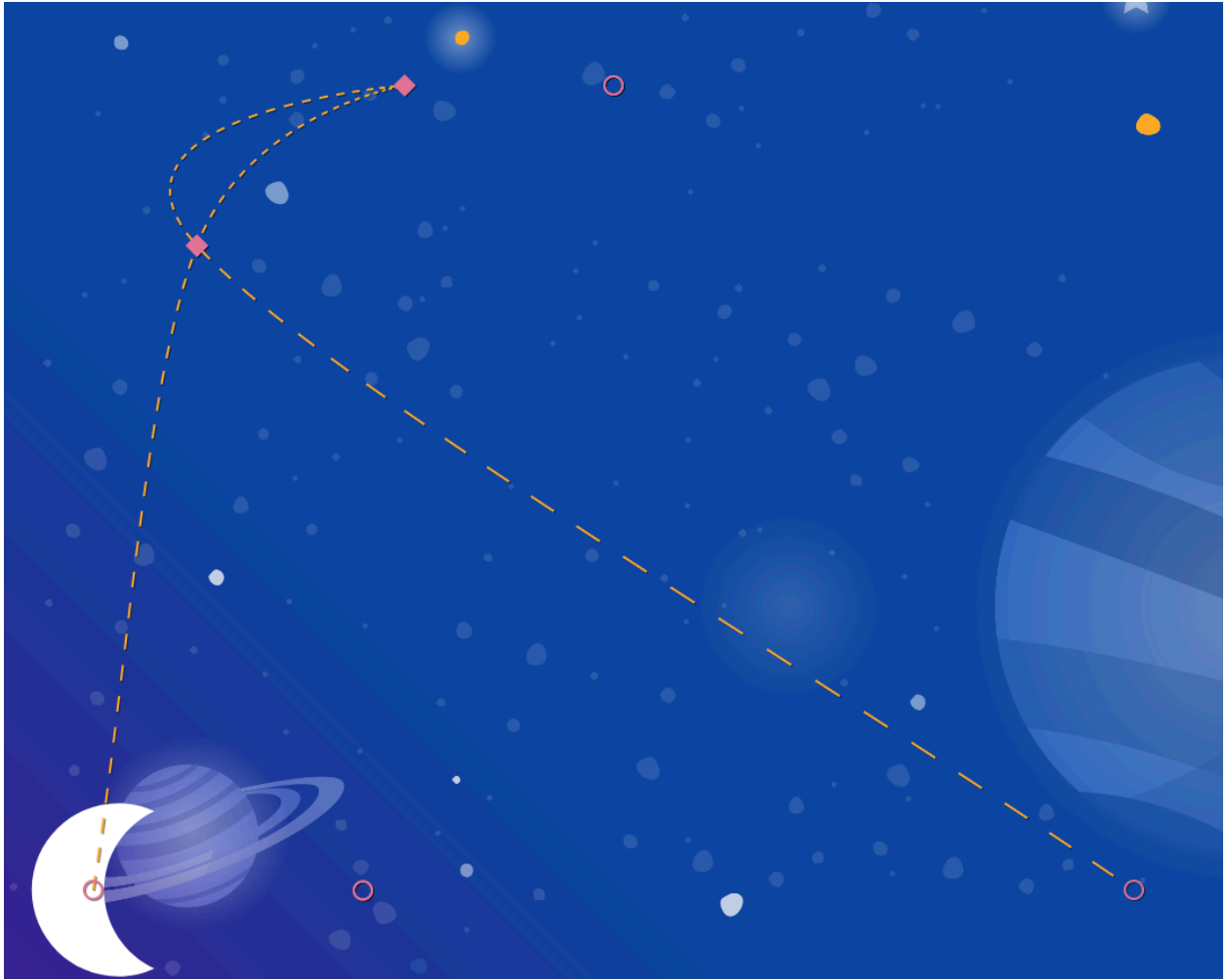
使用多個 `KeyPositon` 標記修改路徑，即可建構複雜的動作。

您可以建構弧線、設定銳角、讓檢視畫面彈跳，或從頭跳到尾。由於 `MotionLayout` 可讓您將多個 `KeyPositions` 套用至同一路徑，因此您可以大幅修改檢視畫面從開始到結束的路徑。

自行探索

在繼續瞭解其他類型的 `KeyFrame` 前，請嘗試在 `KeyFrameSet` 中新增更多 `KeyPositions`，看看只使用 `KeyPosition` 可以建立哪些效果。

以下範例說明如何建構複雜路徑，在動畫期間來回移動。



step4.xml

```
<!-- Complex paths example: Dancing moon -->
<KeyFrameSet>
  <KeyPosition
    motion:framePosition="25"
    motion:motionTarget="@id/moon"
    motion:keyPositionType="parentRelative"
    motion:percentY="0.6"
    motion:percentX="0.1"
  />
  <KeyPosition
    motion:framePosition="50"
    motion:motionTarget="@id/moon"
    motion:keyPositionType="parentRelative"
    motion:percentY="0.5"
    motion:percentX="0.3"
  />
  <KeyPosition
    motion:framePosition="75"
    motion:motionTarget="@id/moon"
    motion:keyPositionType="parentRelative"
    motion:percentY="0.6"
    motion:percentX="0.1"
  />
</KeyFrameSet>
```

探索完 `KeyPosition` 後，請在下一個步驟中繼續瞭解其他類型的 `KeyFrames`。

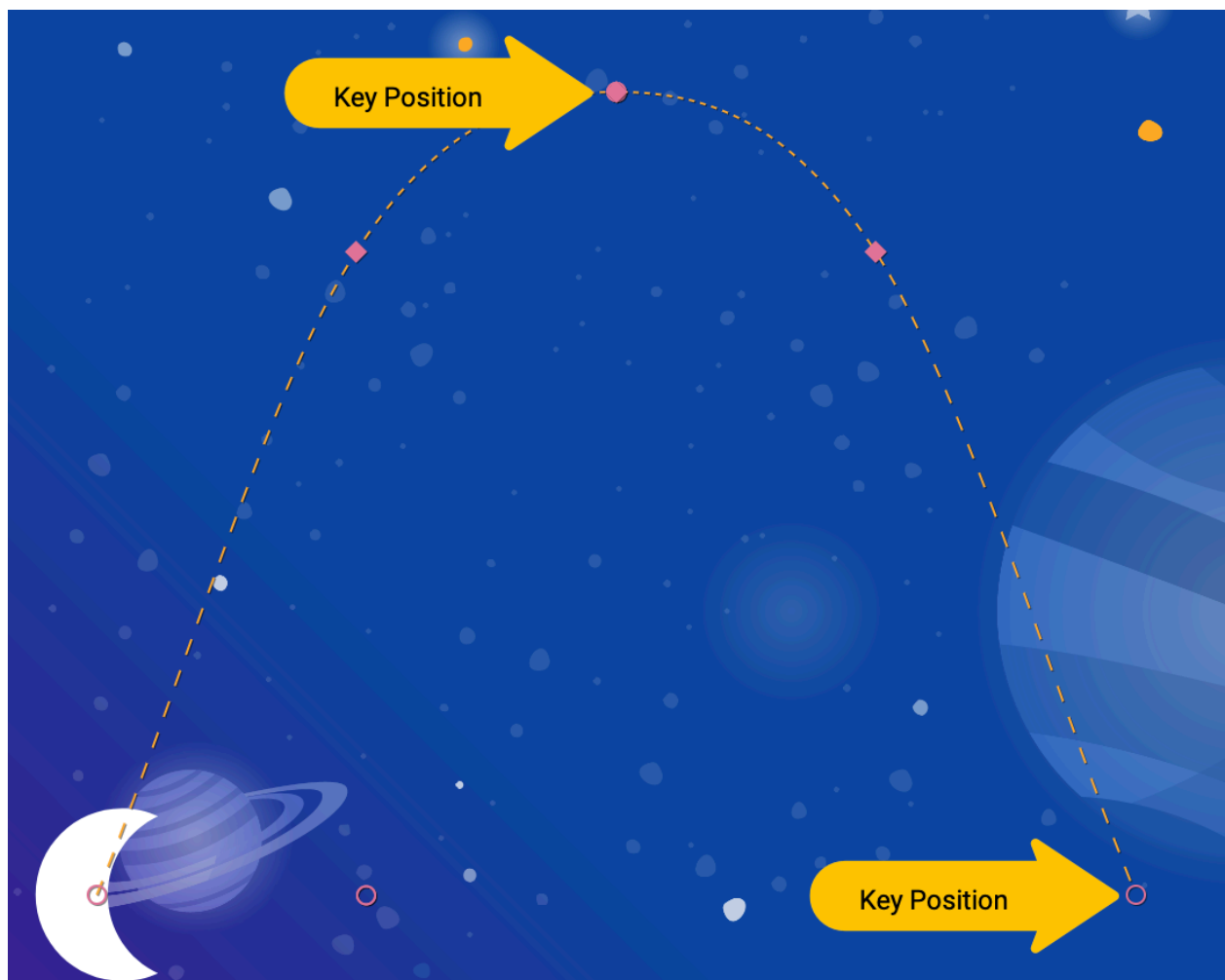
8. 在動作期間變更屬性

建構動態動畫通常是指在動畫進行時，變更檢視區塊的 `size`、`rotation` 或 `alpha`。MotionLayout 支援使用 `KeyAttribute` 為任何檢視畫面上的許多屬性建立動畫。

在這個步驟中，您會使用 `KeyAttribute` 縮放及旋轉月球。您也會使用 `KeyAttribute` 延遲顯示文字，直到月亮幾乎完成旅程為止。

步驟 1：使用 `KeyAttribute` 調整大小和旋轉

1. 開啟 `xml/step5.xml`，其中包含您在上一個步驟中建構的相同動畫。為了呈現多樣性，這個畫面使用不同的太空圖片做為背景。
2. 如要讓月球放大並旋轉，請在 `KeyFrameSet` 中的 `keyFrame="50"` 和 `keyFrame="100"` 處新增兩個 `KeyAttribute` 標記。



`step5.xml`

```

<!-- TODO: Add KeyAttributes to rotate and resize @id/moon -->

<KeyAttribute
    motion:framePosition="50"
    motion:motionTarget="@id/moon"
    android:scaleY="2.0"
    android:scaleX="2.0"
    android:rotation="-360"
/>
<KeyAttribute
    motion:framePosition="100"
    motion:motionTarget="@id/moon"
    android:rotation="-720"
/>

```

如要使用 Motion Editor 完成這個步驟，請在總覽面板中選取轉場效果，然後以新增 `KeyPosition` 的方式新增 `KeyAttribute`。

如要為同一個 `KeyAttribute` 新增多個屬性，請在時間軸中點選



(菱形)，然後在屬性面板中編輯 `KeyAttribute`

這些 `KeyAttributes` 會套用至動畫的 50% 和 100%。第一個 `KeyAttribute` (50%) 會出現在圓弧頂端，導致檢視畫面大小加倍，並旋轉 -360 度 (或一整圈)。第二個 `KeyAttribute` 會完成第二次旋轉，達到 -720 度 (兩個完整圓圈)，並將大小縮回正常，因為 `scaleX` 和 `scaleY` 值預設為 1.0。

與 `KeyPosition` 類似，`KeyAttribute` 會使用 `framePosition` 和 `motionTarget` 指定套用 `KeyFrame` 的時機，以及要修改哪個檢視區塊。`MotionLayout` 會在 `KeyPositions` 之間插值，建立流暢的動畫。

`KeyAttributes` 支援可套用至所有檢視區塊的屬性。支援變更基本屬性，例如 `visibility`、`alpha` 或 `elevation`。您也可以變更旋轉角度 (如這裡所示)、使用 `rotateX` 和 `rotateY` 在三維空間中旋轉、使用 `scaleX` 和 `scaleY` 縮放大小，或在 X、Y 或 Z 軸上平移檢視畫面位置。

單一 `KeyAttribute` 可同時修改多項屬性。

在這個動畫中，`keyFrame="50"` 的 `KeyAttribute` 會設定 `scaleX`、`scaleY` 和 `rotation`。`MotionLayout` 會同時修改這三項屬性。

步驟 2：延遲顯示製作人員名單

這個步驟的目標之一是更新動畫，讓片尾文字在動畫大致完成後才顯示。

1. 如要延遲顯示製作人員名單，請再定義一個 `KeyAttribute`，確保 `alpha` 在 `keyPosition="85"` 之前維持 0。`MotionLayout` 仍會從 0 到 100 的 Alpha 值平滑轉場，但會在動畫的最後 15% 執行這項操作。

step5.xml

```
<!-- TODO: Add KeyAttribute to delay the appearance of @id/credits -->

<KeyAttribute
    motion:framePosition="85"
    motion:motionTarget="@id/credits"
    android:alpha="0.0"
/>
```

這項 `KeyAttribute` 會在動畫的前 85% 期間，將 `@id/credits` 的 `alpha` 保持在 0.0。由於動畫是從 Alpha 值 0 開始，因此在動畫的前 85% 時間內，元素會處於隱藏狀態。

這項 `KeyAttribute` 的最終效果是片尾會在動畫結尾附近顯示。這樣一來，月亮就會在畫面右下角緩緩落下，看起來就像是與月亮同步。

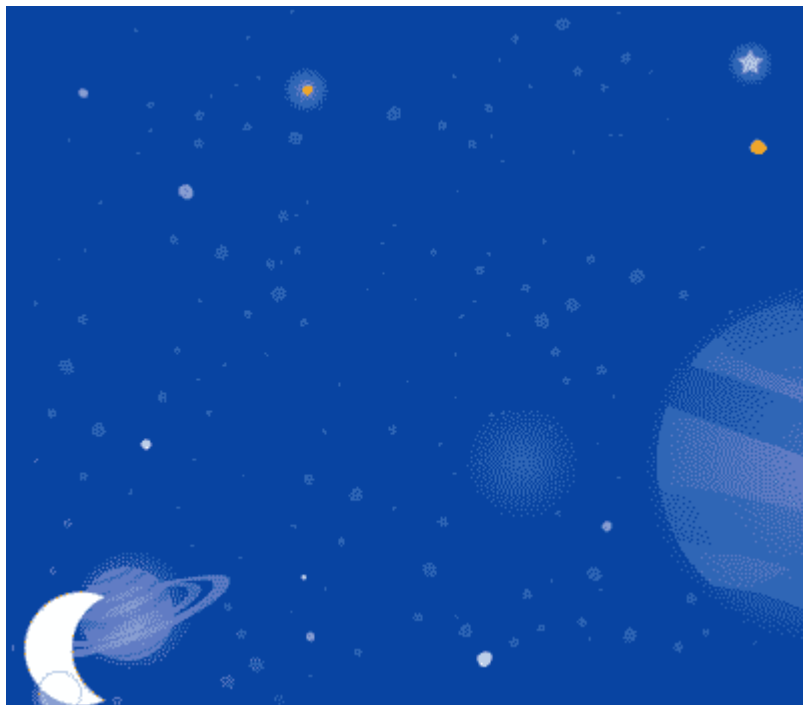
在另一個檢視區塊以這種方式移動時，延遲一個檢視區塊的動畫，即可建構令人驚豔的動畫，讓使用者感覺充滿動態效果。

`KeyAttribute` 絕不會變更檢視畫面在開始或結束位置的外觀。

如果檢視區塊在 `framePosition="100"` (或動畫的 100%) 旋轉、平移或縮放，就會「跳躍」至最終值。如要變更開始或結束狀態，請修改 `ConstraintSet` 中的 `Constraints`。

馬上試試

1. 再次執行應用程式，然後前往**步驟 5**，查看動畫的實際運作情形。點選月亮後，月亮就會沿著路徑從起點移動到終點，並經過 `KeyFrameSet` 中指定的每個 `KeyAttribute`。



由於你將月球旋轉了兩圈，現在月球會向後翻轉兩次，且片尾會延遲顯示，直到動畫快要結束時才會出現。

自行探索

在繼續處理最後一種 `KeyFrame` 之前，請先嘗試修改 `KeyAttributes` 中的其他標準屬性。舉例來說，請嘗試將 `rotation` 變更為 `rotationX`，看看會產生什麼動畫。

以下列出可嘗試的標準屬性：

- `android:visibility`
 - `android:alpha`
 - `android:elevation`
 - `android:rotation`
 - `android:rotationX`
 - `android:rotationY`
 - `android:scaleX`
 - `android:scaleY`
 - `android:translationX`
 - `android:translationY`
 - `android:translationZ`
-

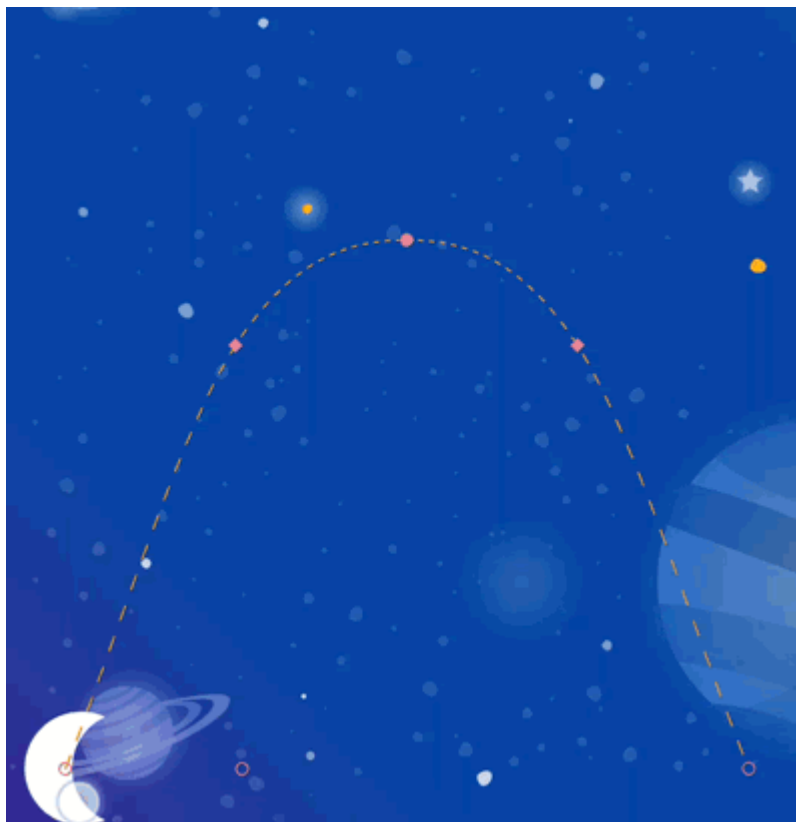
步驟 9 · 約 4 分鐘

9. 變更自訂屬性

豐富動畫包括變更檢視區塊的顏色或其他屬性。 `MotionLayout` 可以使用 `KeyAttribute` 變更上一個工作列出的任何標準屬性，但您可以使用 `CustomAttribute` 指定任何其他屬性。

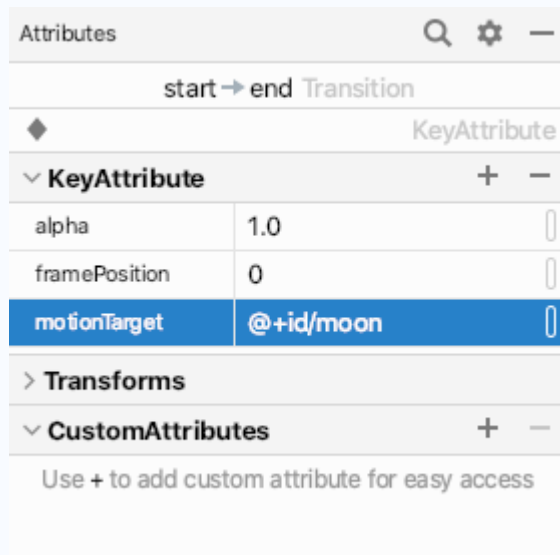
`CustomAttribute` 可用於設定任何有設定器的值。舉例來說，您可以使用 `CustomAttribute` 在 `View` 上設定 `backgroundColor`。 `MotionLayout` 會使用反射尋找設定器，然後重複呼叫設定器，為檢視區塊製作動畫。

在這個步驟中，您將使用 `CustomAttribute` 在月球上設定 `colorFilter` 屬性，建構如下所示的動畫。



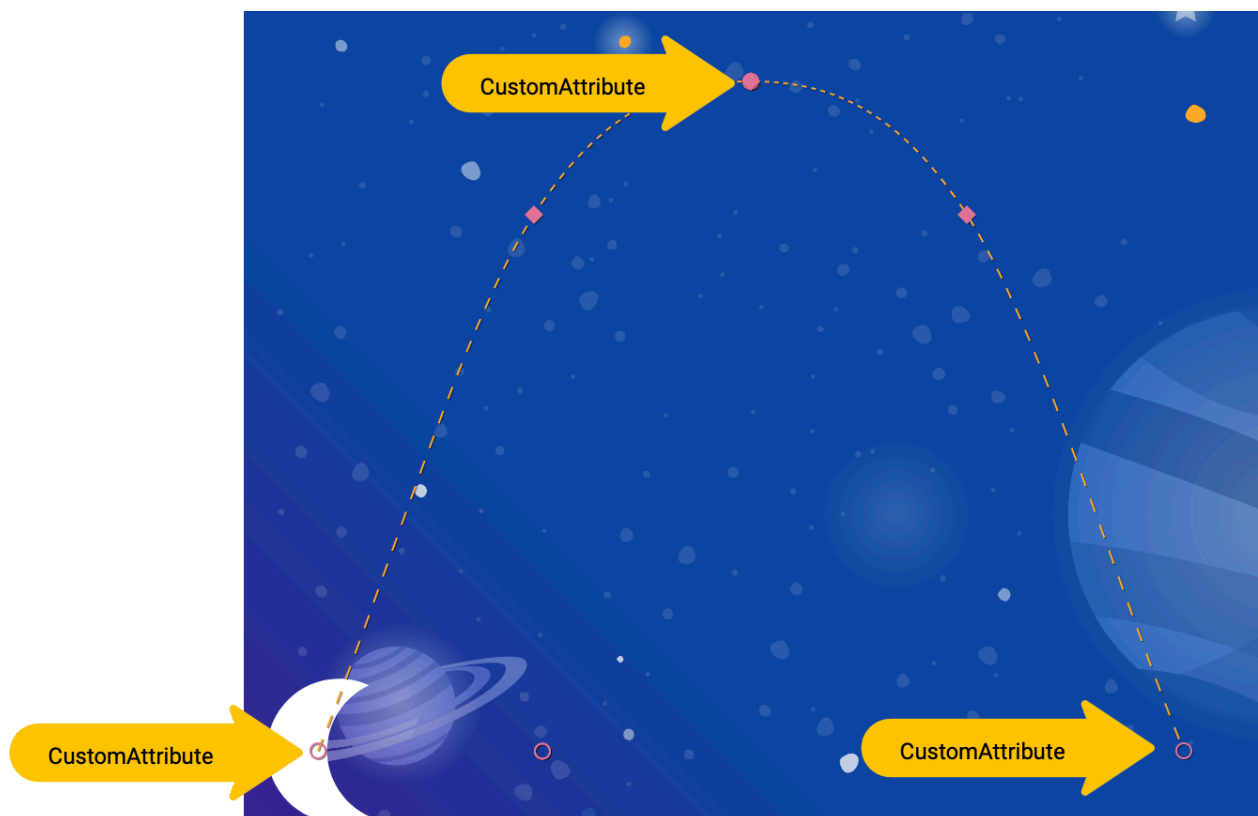
如要在 Motion Editor 中新增自訂屬性，請先建立 KeyAttribute，然後在屬性面板中編輯。

屬性面板中新增了「CustomAttributes」部分，並提供「+」按鈕，可新增自訂屬性。



定義自訂屬性

1. 如要開始使用，請開啟 `xml/step6.xml`，其中包含您在上一個步驟中建構的動畫。
2. 如要變更月亮的顏色，請在 `KeyFrameSet` 中於 `keyFrame="0"`、`keyFrame="50"` 和 `keyFrame="100"`。新增兩個 `KeyAttribute`，並在其中加入 `CustomAttribute`：



step6.xml

```

<!-- TODO: Add Custom attributes here -->
<KeyAttribute
    motion:framePosition="0"
    motion:motionTarget="@id/moon">
    <CustomAttribute
        motion:attributeName="colorFilter"
        motion:customColorValue="#FFFFFF"
    />
</KeyAttribute>
<KeyAttribute
    motion:framePosition="50"
    motion:motionTarget="@id/moon">
    <CustomAttribute
        motion:attributeName="colorFilter"
        motion:customColorValue="#FFB612"
    />
</KeyAttribute>
<KeyAttribute
    motion:framePosition="100"
    motion:motionTarget="@id/moon">
    <CustomAttribute
        motion:attributeName="colorFilter"
        motion:customColorValue="#FFFFFF"
    />
</KeyAttribute>

```

在 `KeyAttribute` 中加入 `CustomAttribute`。系統會在 `KeyAttribute` 指定的 `framePosition` 套用 `CustomAttribute`。

在 `CustomAttribute` 中，您必須指定 `attributeName` 和要設定的值。

- `motion:attributeName` 是這個自訂屬性會呼叫的設定器名稱。在本例中，系統會呼叫 `setColorFilter` 上的 `Drawable`。
- `motion:custom*Value` 是名稱中註記的類型自訂值，在本例中，自訂值是指定的顏色。

自訂值可以是下列任一類型：

- 顏色
- 整數
- 浮點值
- 字串
- 維度
- 布林值

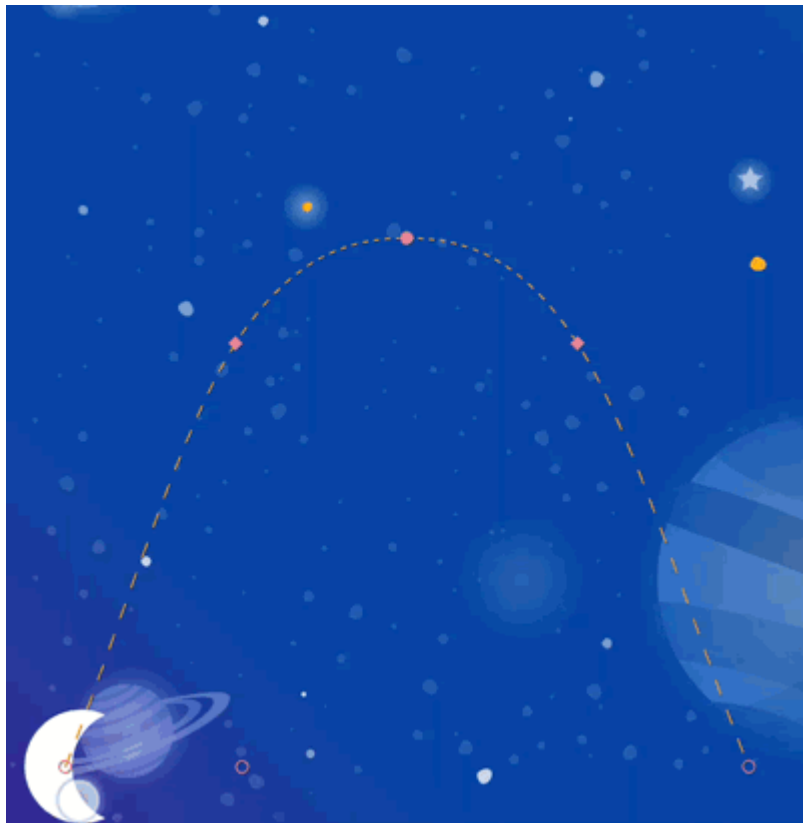
使用這個 API，`MotionLayout` 可以為任何檢視區塊上提供設定器的項目製作動畫。

自訂檢視區塊可使用 `CustomAttribute` 製作動畫。

只要 `MotionLayout` 能找到採用正確型別的設定器，就能為值之間的變更建立動畫效果。

馬上試試

1. 再次執行應用程式，然後前往「步驟 6」，查看動畫的實際運作情形。點選月亮後，月亮就會沿著路徑從起點移動到終點，並經過 `KeyFrameSet` 中指定的每個 `KeyAttribute`。



新增更多 `KeyFrames` 後，`MotionLayout` 會將月亮的移動路徑從直線改為複雜的曲線，並在動畫中途加入雙重後空翻、大小調整和顏色變化。

在實際動畫中，您通常會同時為多個檢視區塊製作動畫，並沿著不同路徑和速度控制其動作。為每個檢視區塊指定不同的 `KeyFrame`，即可編排豐富的動畫，使用 `MotionLayout` 為多個檢視區塊製作動畫。

10. 拖曳事件和複雜路徑

在這個步驟中，您將瞭解如何搭配複雜路徑使用 `OnSwipe`。到目前為止，月亮的動畫是由 `OnClick` 監聽器觸發，並以固定時間長度執行。

如要使用 `OnSwipe` 控制路徑複雜的動畫 (例如您在最後幾個步驟中建構的月球動畫)，必須瞭解 `OnSwipe` 的運作方式。

步驟 1：探索 `OnSwipe` 行為

1. 開啟 `xml/step7.xml` 並找出現有的 `OnSwipe` 宣告。

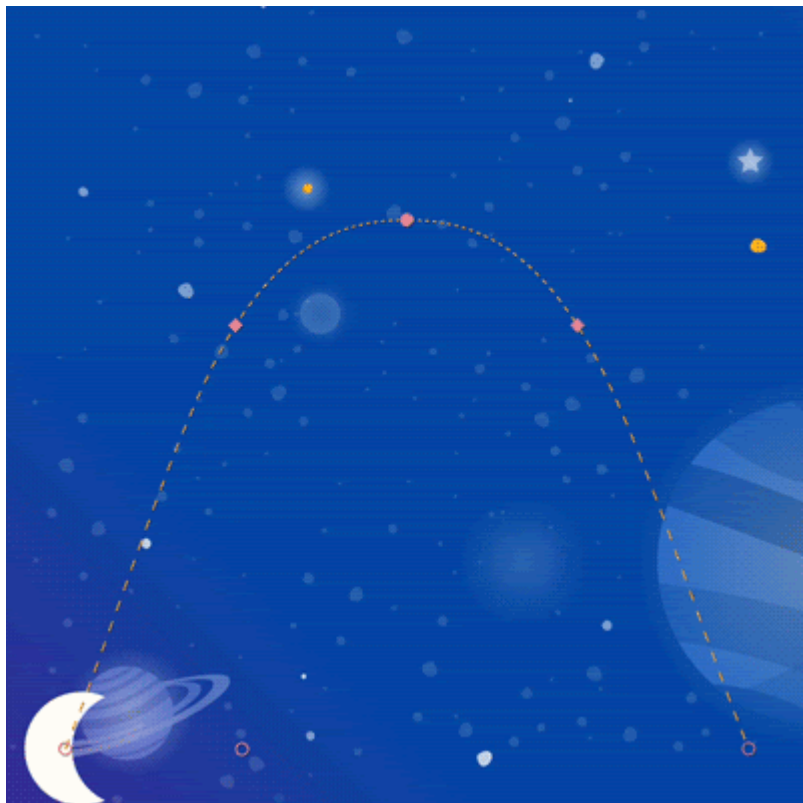
`step7.xml`

```
<!-- Fix OnSwipe by changing touchAnchorSide ->

<OnSwipe
    motion:touchAnchorId="@id/moon"
    motion:touchAnchorSide="bottom"
/>
```

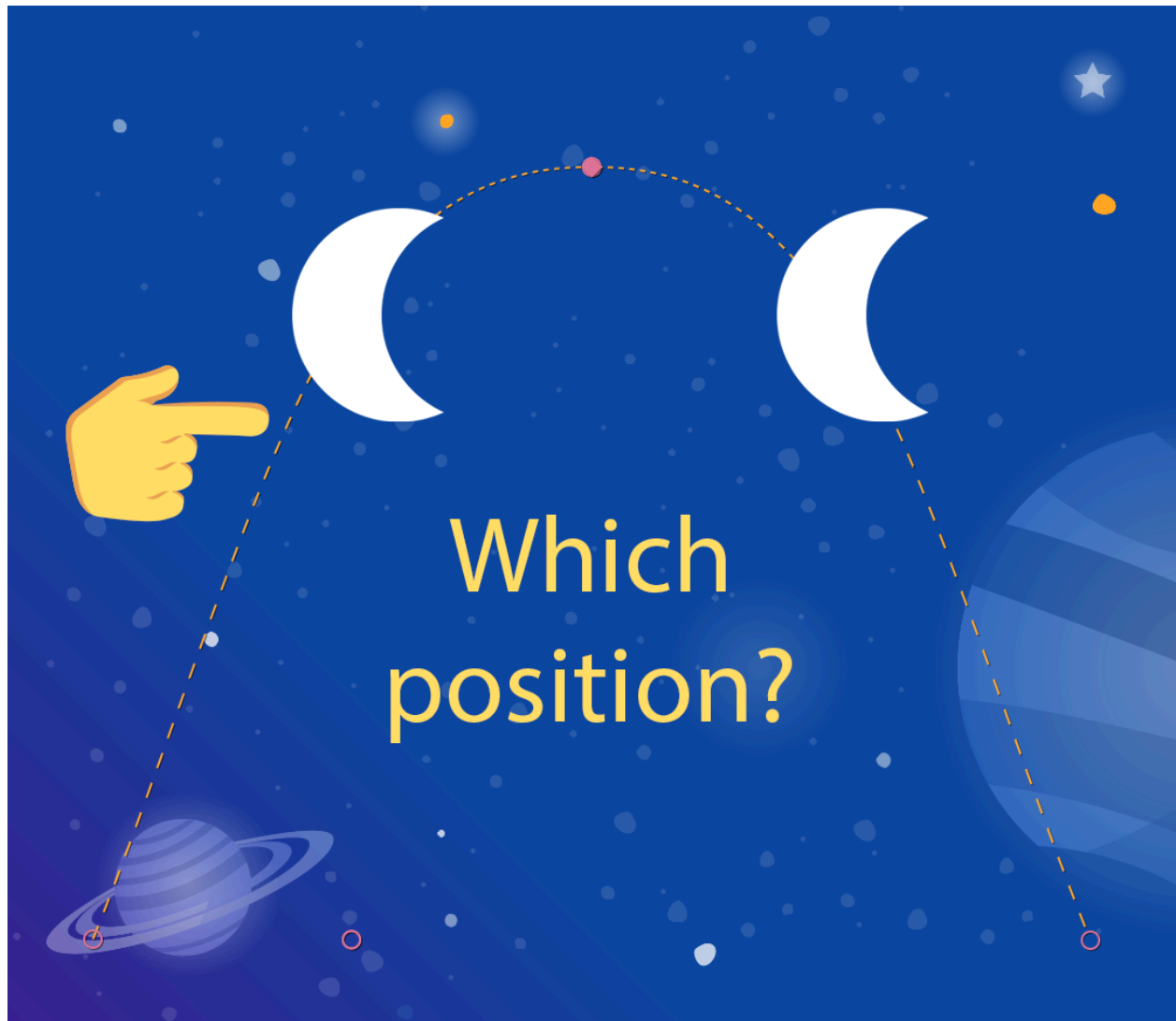
2. 在裝置上執行應用程式，然後前往步驟 7。沿著圓弧路徑拖曳月亮，看看是否能產生平滑的動畫。

執行這項動畫時，效果不太好。月亮到達弧線頂端後，就會開始跳動。



如要瞭解這個錯誤，請考慮使用者觸控圓弧頂端下方時會發生什麼情況。由於 `OnSwipe` 標記具有 `motion:touchAnchorSide="bottom"`，`MotionLayout` 會盡量讓手指與檢視區塊底部的距離在整個動畫中保持不變。

不過，由於月亮底部不一定會朝同一個方向移動，而是會先上升再下降，因此當使用者剛通過弧線頂端時，`MotionLayout` 就不知道該怎麼做。考量到這一點，由於您追蹤的是月球底部，使用者觸控這個位置時，月球底部應該放在哪裡？



步驟 2：使用右側

為避免發生這類錯誤，請務必選擇在整個動畫期間，一律朝單一方向進展的 `touchAnchorId` 和 `touchAnchorSide`。在這項動畫中，月球的 `right` 側和 `left` 側都會朝同一方向移動。不過，`bottom` 和 `top` 都會反向。OnSwipe 嘗試追蹤時，如果方向改變，就會感到困惑。

如要在 Motion Editor 中完成這些步驟，請在總覽面板中選取轉場效果，並在屬性面板中編輯 OnSwipe。

1. 如要讓這個動畫追蹤觸控事件，請將 `touchAnchorSide` 變更為 `right`。

step7.xml

```
<!-- Fix OnSwipe by changing touchAnchorSide -->

<OnSwipe
    motion:touchAnchorId="@id/moon"
    motion:touchAnchorSide="right"
/>
```

傳遞至 `OnSwipe` 的 `touchAnchorSide` 必須在整個動畫中朝單一方向移動。

如果錨定側邊反向移動或暫停，`MotionLayout` 會感到困惑，無法順暢移動。

在某些動畫中，沒有任何檢視區塊具有適當的 `touchAnchorSide`。

如果每個邊都沿著複雜路徑移動，或是檢視區塊以會產生令人意外動畫的方式調整大小，就可能發生這種情況。在這種情況下，建議您新增可追蹤的隱形檢視區塊，並遵循較簡單的路徑。

步驟 3：使用 `dragDirection`

你也可以將 `dragDirection` 與 `touchAnchorSide` 結合，讓側軌朝與平常不同的方向移動。`touchAnchorSide` 仍只能朝一個方向移動，但您可以告知 `MotionLayout` 要追蹤哪個方向。舉例來說，您可以保留 `touchAnchorSide="bottom"`，但新增 `dragDirection="dragRight"`。這會導致 `MotionLayout` 追蹤檢視畫面底部的位置，但只會在向右移動時考量其位置 (系統會忽略垂直移動)。因此，即使底部會上下移動，`OnSwipe` 仍會正確地產生動畫。

1. 更新 `OnSwipe`，正確追蹤月球的移動軌跡。

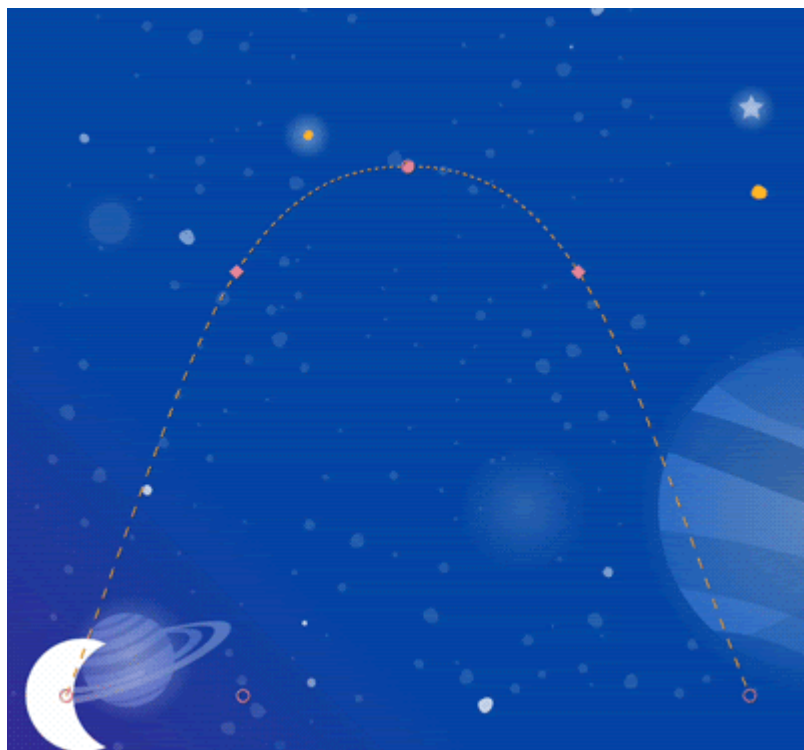
step7.xml

```
<!-- Using dragDirection to control the direction of drag tracking -->

<OnSwipe
    motion:touchAnchorId="@id/moon"
    motion:touchAnchorSide="bottom"
    motion:dragDirection="dragRight"
/>
```

馬上試試

1. 再次執行應用程式，然後嘗試將月亮拖曳到整個路徑。即使路徑複雜，`MotionLayout` 也能因應滑動事件推進動畫。



11. 使用程式碼執行動作

搭配 `CoordinatorLayout` 使用 `MotionLayout`，即可建構豐富的動畫。在這個步驟中，您將使用 `MotionLayout` 建構可收合的標題。

步驟 1：探索現有程式碼

1. 如要開始使用，請開啟 `layout/activity_step8.xml`。
2. 在 `layout/activity_step8.xml` 中，您會看到已建構的 `CoordinatorLayout` 和 `AppBarLayout`。

`activity_step8.xml`

```
<androidx.coordinatorlayout.widget.CoordinatorLayout
    ...>
    <com.google.android.material.appbar.AppBarLayout
        android:id="@+id/appbar_layout"
        android:layout_width="match_parent"
        android:layout_height="180dp">
        <androidx.constraintlayout.motion.widget.MotionLayout
            android:id="@+id/motion_layout"
            ... >
            ...
        </androidx.constraintlayout.motion.widget.MotionLayout>
    </com.google.android.material.appbar.AppBarLayout>

    <androidx.core.widget.NestedScrollView
        ...
        motion:layout_behavior="@string/appbar_scrolling_view_behavior" >
        ...
    </androidx.core.widget.NestedScrollView>
</androidx.coordinatorlayout.widget.CoordinatorLayout>
```

這個版面配置使用 `CoordinatorLayout` 在 `NestedScrollView` 和 `AppBarLayout` 之間分享捲動資訊。因此，當 `NestedScrollView` 向上捲動時，會將這項變更告知 `AppBarLayout`。這就是在 Android 上實作這類可摺疊工具列的方式，文字的捲動會與摺疊式標題「協調」。

`@id/motion_layout` 指向的動作場景與上一個步驟中的動作場景類似。不過，為讓它能與 `CoordinatorLayout` 搭配運作，我們移除了 `OnSwipe` 宣告。

3. 執行應用程式，然後前往步驟 8。捲動文字時，月亮不會移動。

步驟 2：讓 `MotionLayout` 捲動

1. 如要讓 `MotionLayout` 檢視區塊在 `NestedScrollView` 捲動時立即捲動，請將 `motion:minHeight` 和 `motion:layout_scrollFlags` 新增至 `MotionLayout`。

`activity_step8.xml`

```
<!-- Add minHeight and layout_scrollFlags to the MotionLayout -->

<androidx.constraintlayout.motion.widget.MotionLayout
    android:id="@+id/motion_layout"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    motion:layoutDescription="@xml/step8"
    motion:motionDebug="SHOW_PATH"
    android:minHeight="80dp"
    motion:layout_scrollFlags="scroll|enterAlways|snap|exitUntilCollapsed" >
```

2. 再次執行應用程式，然後前往**步驟 8**。您會發現 `MotionLayout` 會在上捲時收合。不過，動畫尚未根據捲動行為進展。

步驟 3：使用程式碼移動動作

1. 開啟 `Step8Activity.kt`。編輯 `coordinateMotion()` 函式，向 `MotionLayout` 說明捲動位置的變化。

Step8Activity.kt

```
// TODO: set progress of MotionLayout based on an AppBarLayout.OnOffsetChangedListener

private fun coordinateMotion() {
    val appBarLayout: AppBarLayout = findViewById(R.id.appbar_layout)
    val motionLayout: MotionLayout = findViewById(R.id.motion_layout)

    val listener = AppBarLayout.OnOffsetChangedListener { unused, verticalOffset ->
        val seekPosition = -verticalOffset / appBarLayout.totalScrollRange.toFloat()
        motionLayout.progress = seekPosition
    }

    appBarLayout.addOnOffsetChangedListener(listener)
}
```

這段程式碼會註冊 `OnOffsetChangedListener`，每次使用者捲動時，系統都會呼叫這個函式，並傳遞目前的捲動偏移量。

`MotionLayout` 支援透過設定進度屬性來搜尋轉場效果。如要在 `verticalOffset` 和百分比進度之間轉換，請除以總捲動範圍。

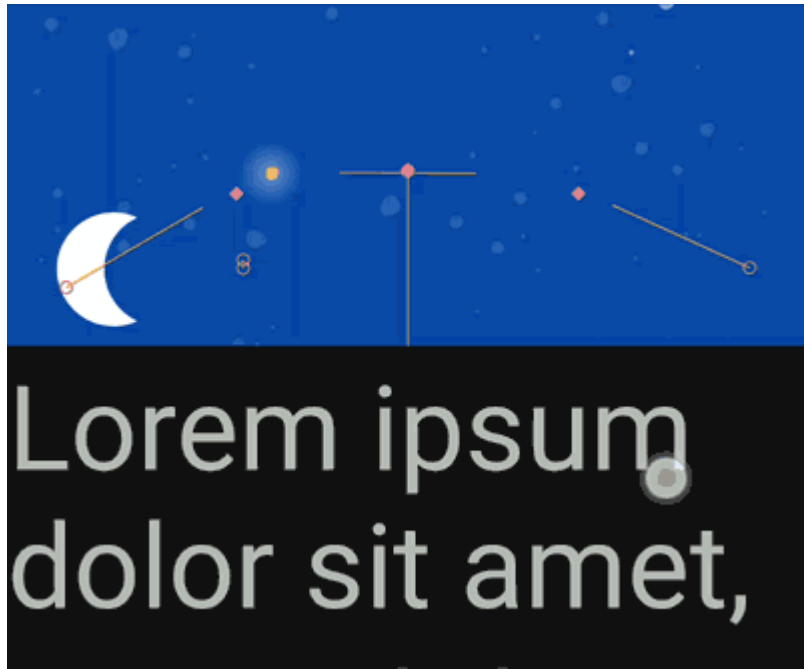
`MotionLayout` 可以在程式碼中跳轉到動畫中的特定時間點。

請設定 `motionLayout.progress`，`MotionLayout` 會立即「跳轉」至指定位置。

舉例來說，如果將進度設為 0.43，`MotionLayout` 會透過動畫跳到 43%。

馬上試試

1. 再次部署應用程式，然後執行「步驟 8」動畫。您會看到 `MotionLayout` 會根據捲動位置推進動畫。



您可以使用 `MotionLayout` 建立自訂動態收合工具列動畫。使用一連串的 `KeyFrames` 即可達到粗體效果。

`AppBarLayout` 不會調整 `MotionLayout` 的大小。

`AppBarLayout` 收合 `MotionLayout` 檢視畫面時，該畫面會部分移出螢幕。不會調整大小，只會向上移動。如果 `MotionLayout` 頂端有限制，動畫結束時就會超出畫面。如要使用 `AppBarLayout`，請確保結尾限制條件固定在父項底部。

步驟 12 · 約 0 分鐘

12. 恭喜

本程式碼研究室介紹了 `MotionLayout` 的基本 API。

如要查看更多 `MotionLayout` 的實際應用範例，請參閱官方[範例](#)。請務必參閱[說明文件](#)！

瞭解詳情

`MotionLayout` 支援更多本程式碼研究室未涵蓋的功能，例如 `KeyCycle`，(可透過重複週期控制路徑或屬性)，以及 `KeyTimeCycle`，(可根據時鐘時間製作動畫)。請參閱範例，瞭解各項功能。

如要查看本課程其他程式碼研究室的連結，請參閱 [Android Kotlin 進階功能程式碼研究室到達網頁](#)。
